Order, Symmetry and the Swap: A Synthesis of Matrix Transposition on Tapes

Hans Brende Claude Fable 5.2 (Anthropic)

November 2026

Abstract

Can a deterministic multitape Turing machine transpose a $d \times d$ matrix with $\lceil \log_2 d \rceil$-bit entries in time linear in the input length L? Sixteen earlier papers of this series study the question without deciding it. This paper is a synthesis of them, a diagnosis of where they stop, and two new theorems.

The synthesis states the main results of the series in one place, with their main hypotheses. The diagnosis is this. For oblivious machines the question is now one about a single function, the best rate of a storage code on the bounded-depth cone relation that is left of a linear-time schedule after a genie has revealed εL bits: a transposer needs rate $1 - \varepsilon$ for every ε, pairing gives rate $\frac{1}{2}$, and for the relations of real machines in general no rate above $\frac{1}{2}$ is achieved and no useful bound below one is proved. Conditions on head motion that ignore the data are met by a machine that transposes nothing. Bounds for codes that ignore the machine fail, because abstract relations with the two properties used so far, bounded degree and paired-distance separation, carry codes of rate tending to one. Counting passes or cuts saturates one scale at a time. The common reason is that the $\log d$ of the folklore algorithm is a sum over address scales, and every tool that applies to all machines looks at one scale or at none; the logarithm is proved only where either the coding or the geometry is tied down.

The theorems concern the swap τ on $V \otimes V$, $V = F^d$, and the code $\ker(\tau - K)$ of matrices that an operator $K = \sum_a A_a \otimes B_a$ transposes correctly. *Order*: for every complete flag of V, the code has dimension at most the sum of the dimensions of the images of the flag under the B_a, whatever the A_a are; so two terms, commuting families, and, in characteristic p with $d = p^m$, all affine index maps stay at the dimension $d(d+1)/2$ of the symmetric matrices. *Symmetry*: if each row of a matrix is a fixed linear function of the matrix seen from that row's position, the same function on each of M intervals of rows, then the matrix ranges over a space of dimension at most $(d^2 + Md)/2$, on a cycle and on a line, with equality for equal intervals, where the bound is the dimension of the coarse $M \times M$ block transposition. The proof intersects the code with its own translate. It works in every characteristic and uses a property of machines that cone relations do not record: translated outputs share their decoder. Variants bound decoders with few modulation frequencies and show that every address digit must be treated asymmetrically, which one unshuffle does for all digits at once. For classes of rows that no single shift respects the bound can fail: at $d = 6$ two such classes carry a code of dimension $25 > 24$.

Both theorems are statements about a linear one-hop model, not about machines, and the question remains open. The coordinate form of the first theorem, its attainment, the pigeonhole lemma, the second theorem on a cycle for arbitrary shift sets and on a line, its attainment for equal intervals, and the twist-class variant are formalized in Lean 4 on the standard axioms. The corollaries that need an algebraically closed field or the structure of p-groups, the copy-rule bound, the modulation and digit corollaries and the passage from flags to bases are proved in the text and not formalized. Exact computations at sizes up to six are reported as numerical remarks.

Contributions. Hans Brende chose the problem, fixed the verification standard, directed the research, and asked for this synthesis. Claude Fable 5.2, a large language model developed by Anthropic, read the sixteen earlier papers and their memos, wrote the synthesis and the diagnosis, developed the mathematics of Sections 5 and 6 and the exact computations of Section 8, directed the Lean formalization, and wrote this paper. Independent instances of the same model refereed the two theorems; one of them contributed Corollary 5.8, the line form of Theorem 6.4 (Theorem 6.9) and the counterexample of Remark 6.10.

Contents

1 Introduction **4**
- 1.1 The problem . . . 4
- 1.2 What this paper does . . . 4
- 1.3 How to read this paper and its formal proofs . . . 5
- 1.4 Related work . . . 6
- 1.5 Organization . . . 6

2 The problem and two models **7**
- 2.1 The question . . . 7
- 2.2 Storage codes on recovery relations . . . 7
- 2.3 The linear one-hop model . . . 8

3 The series in one place **8**
- 3.1 The question, passes and the coding formulation . . . 9
- 3.2 What head motion must do . . . 9
- 3.3 Depth reduction . . . 11
- 3.4 Storage codes: what is proved . . . 11
- 3.5 Storage codes: what is false . . . 12
- 3.6 Adaptive machines . . . 13

4 Where the wall is **13**
- 4.1 What is left is one function . . . 13
- 4.2 Three kinds of argument and their limits . . . 14
- 4.3 Why every general tool saturates . . . 15
- 4.4 What the missing statement must be . . . 16
- 4.5 What a construction cannot avoid . . . 16
- 4.6 The rule and the relation . . . 17

5 Order: the flag bound **17**
- 5.1 The theorem . . . 17
- 5.2 Corollaries . . . 18
- 5.3 What the theorem says . . . 21

6 Symmetry: the translate pigeonhole **21**
- 6.1 The lemma . . . 21
- 6.2 Class-uniform rules . . . 22
- 6.3 On a line . . . 24
- 6.4 Twists, digits and composition . . . 25
- 6.5 What the theorem says . . . 27

7 The construction side **27**
7.1 Mechanisms that fail . . . 27
7.2 A reading for tapes . . . 28
7.3 What a linear-time machine would need . . . 28
7.4 From the one-hop model to machines . . . 29

8 Small sizes **29**
8.1 Method . . . 29
8.2 Results . . . 30

9 Open problems **31**

A Index of Lean declarations **32**

B Building and verifying the formalization **33**

1 Introduction

1.1 The problem

Can a deterministic multitape Turing machine transpose a $d \times d$ matrix with $\lceil \log_2 d \rceil$-bit entries in time linear in the input length $L = d^2 \lceil \log_2 d \rceil$? We call the affirmative statement (T); Section 2.1 states it precisely. The folklore algorithm takes $O(L \log d)$ steps. Transposition is the data-movement step of fast integer multiplication on tapes, and by a reduction of Harvey and van der Hoeven [34] a linear-time multiplier would give (T). So a proof that (T) fails would be the first superlinear lower bound for off-line multiplication on multitape machines, and a proof that it holds would be a new way of moving data.

Sixteen papers and eight memos of this series have studied the question since October 2026. They contain reductions, restricted impossibility theorems, constructions of codes and many formal proofs. They do not contain the answer, and a reader who starts today has to read all of them to learn where the difficulty is. This paper is written for that reader.

1.2 What this paper does

It does three things.

A synthesis (Section 3). We state the main results of the series in one place, by theme: the reductions, what head motion must do, depth reduction, what is proved about storage codes, what is false about them, and adaptive machines. Nothing is reproved. Every statement is cited, with its main hypotheses.

A diagnosis (Section 4). For oblivious machines the series has reduced (T) to one function: the best rate $r(\varepsilon)$ of a storage code on the bounded-depth cone relations that remain of linear-time schedules when a genie reveals εL bits. A transposer needs $r(\varepsilon) \geq 1 - \varepsilon$ for every ε. The matching of each stored symbol with one partner gives $r \geq \frac{1}{2}$, and whether r exceeds $\frac{1}{2}$ at all on the relations of real machines is open in both directions. Three kinds of argument have reached known limits:

- conditions on motion that ignore the data, because a machine that transposes nothing satisfies the cut inequalities behind them; conditions that do see the outputs are single-scale, and one cut serves every pair of row sets;
- bounds for codes that ignore the machine, because abstract relations with the two properties used so far, bounded degree and paired-distance separation, carry codes of rate tending to one;
- counting passes, because it saturates one scale at a time.

They stop for one reason. The logarithm of the folklore bound is a sum over $\log_2 d$ address scales, each costing L, and every tool of the series that applies to all machines looks at one scale or at none; the logarithm is proved only where the coding or the geometry is tied down. The missing statement has to tie what is known to where it can be, for all schedules, and the coding properties proved so far are properties of the cone *relation*, which forgets that neighbouring outputs of a machine are produced by the same rule.

Two theorems (Sections 5 and 6). We prove two rigidity theorems for the swap in a model where the code and its decoder are visible together. Let $V = F^d$ over any field, let τ be the swap on $V \otimes V$, that is, $X \mapsto X^{\mathsf{T}}$ on matrices, and let $K = \sum_a A_a \otimes B_a$ act by $X \mapsto \sum_a A_a X B_a^{\mathsf{T}}$. The code $\ker(\tau - K)$ is the set of matrices that K transposes correctly. The symmetric matrices, for $K = 1$, have dimension $d(d+1)/2$.

- *Order (Theorem FL, Theorem 5.1).* For every complete flag $W_1 \subset \cdots \subset W_d$ of V, with $g(i) = \dim \sum_a B_a(W_i)$,

$$\dim \ker(\tau - K) \le \sum_{i=1}^{d} g(i),$$

  whatever the A_a are. So two Kronecker terms never beat the symmetric matrices, over any field; neither do commuting B_a, for instance any convolutions of any number of rows; and neither does any combination of rotations, reflections, dilations and decimations when $d = p^m$ in characteristic p. The bound is attained for every profile g.

- *Symmetry (Theorem TP, Theorem 6.4).* For any three operators on an N-dimensional space, $\dim \ker(\tau - K) \le \frac{1}{2}(N + \dim \ker[\tau, T] + \operatorname{rank}[K, T])$, an elementary lemma. For the swap, $\dim \ker[\tau, T] \ge d$ always. If row c of X is a fixed linear function of X with its columns rotated by c, the same function for all c in each of M intervals, then

$$\dim\{X\} \le \frac{d^2 + Md}{2},$$

  on a cycle and on a line, with equality for equal intervals: there the largest code has the dimension of the coarse $M \times M$ block transposition. For M equal intervals flags give only rate $1 - 1/(2M)$. Variants bound decoders with s modulation frequencies by rate $s/(s+1)$ and show that every address digit must be treated asymmetrically. The hypothesis on the classes is needed: for $d = 6$, two classes with no almost-period carry a code of dimension $25 > 24$ (Proposition 8.2), found by a SAT solver and checked in Lean.

In words: to beat the pairing value, the decoder may have neither a hidden order nor a symmetry, on either index, that the swap does not share. The second theorem uses exactly the feature of machines that cone relations forget.

What the theorems are not. They are statements about the linear one-hop model. They are not lower bounds for machines, and (T) remains open. Section 7 lists what they exclude and the three steps that would be needed to carry them to machines; Section 8 gives exact values at small sizes; Section 9 states the problems.

1.3 How to read this paper and its formal proofs

The synthesis and the diagnosis contain no numbered results; they cite. The numbered results of Sections 5 and 6 each carry a status line.

- "Lean:" names the formal counterpart in the library `lean/SwapRigidity/`; the Lean statement is authoritative, and the text gives an outline.
- "Lean (partial):" names what is formalized and says what is not.
- "Not formalized" is followed by a complete proof.

Formalized in Lean 4 with Mathlib, on the standard axioms only: the coordinate form of Theorem FL, its attainment and the corollary for a pair of flags; the pigeonhole lemma, its composition property and the lower bound $\dim\ker[\tau, T] \geq d$; Theorem TP on a cycle with an arbitrary shift set, its attainment for equal pieces, and its line form; and the twist-class theorem. Not formalized, and proved in the text: the passage from flags to adapted bases; the corollaries of Theorem FL that need an algebraically closed field, the structure of p-groups or an arithmetic estimate (Corollaries 5.5 and 5.7 to 5.9); the equality case of Proposition 6.2; the copy-rule bound; Corollary 6.12; Corollary 6.13 apart from the abstract lemma it rests on; and the translation between class-uniform rules and operators $\sum_a A_a \otimes B_a$. Numerical observations appear only in remarks labelled as numerical and in Section 8, and no proof uses them. Appendix A indexes the declarations and Appendix B explains how to check them.

1.4 Related work

The series. Section 3 is the account of the earlier papers. The closest of them to the new theorems are the eighth, which states the coding problem for linear machines as relative rigidity of the transposition matrix [2]; the second and fourth, which exclude locally affine head motion [20, 4]; and the seventh, whose elimination lemmas and Cayley corollary are the storage-code relatives of Theorems 5.1 and 6.4 [9].

Matrix equations with a transpose. Equations in which X and X^{T} both occur have been studied as $\star$-Sylvester and $\top$-Stein equations. The solution space of $AX + X^{\star}B = 0$ is described through the Kronecker canonical form of a pencil [29]; that of $XA + AX^{\mathsf{T}} = 0$ is found in [28] and applied to congruence orbits; unique solvability of $AXB + CX^{\star}D = E$ is characterized in [30], and of $X = AX^{\mathsf{T}}B + C$ in [26]. Generalized Schur decompositions are used for solvability and numerical solution [27]. Theorem 5.1 uses triangular forms for a different purpose, a dimension bound in the singular case with one side unconstrained. We found no statement of Corollary 5.5, of the bound for an arbitrary number of terms with one triangularizable side, or of any bound on the rank of the commutation matrix [36] minus a structured sum of Kronecker products. Our search was limited to abstracts and indexes, and we claim no priority for statements that specialists may regard as folklore.

Commutators and network codes. Lemma 6.1 is the fact that a commutator has rank at most twice the rank of either factor; we know no source that states it in the form used here and we give its proof. Transposition as a multi-commodity network-coding instance goes back to [1]. Network codes whose local kernels are sums of cyclic shifts have been studied for multicast [41]; we found nothing on shift-covariant decoders for transposition.

1.5 Organization

Section 2 fixes the problem and the two models. Section 3 is the synthesis and Section 4 the diagnosis. Sections 5 and 6 prove the two theorems. Section 7 is the construction side, Section 8 the exact computations and Section 9 the open problems.

2 The problem and two models

2.1 The question

Throughout, $d \geq 1$ is the side length of a square matrix, $b(d) = \max(1, \lceil \log_2 d \rceil)$ is the entry width, and $L = d^2 b(d)$ is the input length. A machine is a deterministic multitape Turing machine with a read-only input tape, a fixed number k of work tapes, a write-only output tape and a fixed finite alphabet. The input is a $d \times d$ matrix with entries in $\{0,1\}^{b(d)}$, written row by row without delimiters; the output is its transpose in the same encoding.

> **(T)** There are a machine M and a constant c such that, for every d and every input matrix, M halts with the transpose on its output tape within $c(L+1)$ steps.

The authoritative statement is the Lean definition `Transpose.LinearTimeTransposable` of the first paper [7], which the series keeps frozen. A machine is *oblivious* if its head positions at each step, the steps at which it writes output and its halting time depend only on the input length; otherwise it is *adaptive*. A *pass* is a maximal stretch of steps on which every head moves monotonically.

The folklore algorithm transposes in $O(L \log d)$ steps by a recursion on address digits: write the row address and the column address of an entry in binary, and exchange them one digit at a time, each exchange costing $O(L)$. An unrefereed preprint [37] claims $O(L(\log d)^{1-\kappa})$ for a small $\kappa > 0$. Harvey and van der Hoeven [34] reduce transposition to integer multiplication on multitape machines, so a linear-time multiplier would give (T), and a proof that (T) fails would be a superlinear lower bound for off-line multiplication on these machines. No superlinear lower bound is known for transposition, nor for off-line multiplication, sorting or the discrete Fourier transform, on machines with two or more work tapes. The known bounds are for one work tape [32, 31], for machines that only move data [40], and for reversal-bounded conservative machines [35]; the superlinear bounds that follow from the separation of deterministic and nondeterministic linear time [38] concern problems that are hard for nondeterministic linear time [33].

2.2 Storage codes on recovery relations

Most of the series is written in the following language, which we use in Sections 3 and 4.

Definition 2.1 (Storage codes)**.** A *recovery relation* on a finite set V assigns to every $v \in V$ a set $N(v) \subseteq V$; it is *loopless* if $v \notin N(v)$ for all v. A *storage code* on it, over an alphabet α, is a set $C \subseteq \alpha^V$ such that for every v the coordinate w_v of a codeword $w \in C$ is a function of $(w_u)_{u \in N(v)}$. Its *rate* is $\log|C| / (|V| \log|\alpha|)$.

A relation in which $u \in N(v)$ exactly when $v \in N(u)$ is a graph, and the largest $\log_{|\alpha|}|C|$ is then the guessing number of Riis [39]; storage codes are dual to index codes with side information N. On a perfect matching the rate is exactly $\frac{1}{2}$: each vertex copies its partner. We call $\frac{1}{2}$ the *pairing rate*.

The *incidence graph* of a relation has two vertices L_p and R_p for every coordinate p, its port as a helper and its port as a receiver, and an edge $L_p R_q$ whenever $p \in N(q)$. The *paired distance* of p is the distance between L_p and R_p in this graph; for a graph it is the length of the shortest odd closed walk through p. Following the fifth paper [18, Definition 2.3], we call a sequence of relations *separated* if, for every fixed r, all but $o(|V|)$ coordinates have paired distance greater than r. A perfect matching has all paired distances infinite, so separation does not exclude the pairing; it excludes gains that come from short cycles.

For an oblivious machine and a set of steps whose values are revealed (a *genie*), the *cone* of an output bit is the set of input bits on which it can still depend through the time–space dependency

graph. The *cone relation* assigns to an input coordinate q the cone of the output bit that must equal it. If the machine transposes, then for some value of the revealed word the inputs that agree with it form a storage code on the cone relation, of high rate when the genie reveals few bits (Section 3.3). The *depth-D ball* of an output bit is the set of input positions read at steps within D dependency steps of the step that writes it, with no genie.

2.3 The linear one-hop model

The two theorems of this paper live in a smaller model, in which the code and the geometry are visible at once. Let F be a field, $V = F^d$, and let τ be the swap $u \otimes w \mapsto w \otimes u$ on $V \otimes V$. Under the identification of $V \otimes V$ with the $d \times d$ matrices $\mathrm{M}_d(F)$, τ is transposition, $X \mapsto X^{\mathsf{T}}$, and an operator

$$K = \sum_a A_a \otimes B_a, \qquad A_a, B_a \in \mathrm{M}_d(F),$$

acts by $X \mapsto \sum_a A_a X B_a^{\mathsf{T}}$. Every operator on $V \otimes V$ has this form for some finite family of terms.

Definition 2.2 (Swap code)**.** `Lean: swapCode, mem_swapCode` [`Flag.lean`]
The *swap code* of $K = \sum_a A_a \otimes B_a$ is

$$\ker(\tau - K) = \Big\{ X \in \mathrm{M}_d(F) : X^{\mathsf{T}} = \sum_a A_a X B_a^{\mathsf{T}} \Big\}.$$

Its *rate* is $\dim \ker(\tau - K)/d^2$. (The Lean definition is the set of matrices on the right; its identification with the kernel of an operator is not used in the formal proofs.)

The code is the set of matrices that K transposes correctly. Read row by row, the equation says that every column of X is a fixed linear function of the rows of X: the stored word is X in row-major order, the receivers want it in column-major order, and K is the decoder. For $K = 1 \otimes 1$ the code is the space of symmetric matrices, of dimension $d(d+1)/2$. We call this the *pairing value*; its rate tends to the pairing rate $\frac{1}{2}$. For $K = \tau$ the code is everything.

Entries of b bits are handled by replacing $V \otimes V$ with $V \otimes V \otimes F^b$ and τ with $\tau \otimes 1$; the flag bounds below then hold with an extra factor b (Remark 5.10), and Theorem 6.4 likewise.

Relative rigidity. A bound $\dim \ker(\tau - K) \le (1 - \rho)d^2$ for all K in a class says that the transposition matrix is ρd^2-rigid relative to that class: τ is not the sum of a member of the class and a matrix of rank below ρd^2. The eighth paper shows that, for machines with linear data paths, the coding half of the series' programme is exactly such a statement, for the bit-level transposition matrix and the class of matrices whose row for each output bit is supported on its depth-D ball in a schedule [2, Theorem 7.5 and Problem 10.6]; this is a relative form of Valiant's rigidity [42]. The classes of this paper are defined by algebraic structure on one tensor side instead of by a sparsity pattern. The other side is arbitrary.

3 The series in one place

This section states the main results of the sixteen earlier papers and their eight memos, grouped by what they are about and not by when they appeared. Nothing is reproved. Each statement carries its citation and its main hypotheses; the cited theorems have further parameters (the number of tapes, the alphabet, the time constant), which we suppress. Table 1 at the end lists the papers in order. A reader who wants only the conclusion can skip to Section 4.

3.1 The question, passes and the coding formulation

The first paper [7] fixes the formal statement of (T), shows that for linear time it agrees with the simplest case of the formulation of Harvey and van der Hoeven [7, Proposition 11.1, a written proof], and proves the following.

- *Passes.* A transposer with at least one work tape makes $\Omega(\log d)$ passes, whatever it does with its data: for oblivious machines with no premise on the running time, and for all machines that run in time polynomial in L [7, Theorems 5.9 and 5.12]. A memo removes the runtime premise for halting machines that are correct on all inputs [14]. This is the only logarithmic lower bound on a resource of the machine itself that the series proves for all machines. It counts passes, not steps, and it is tight: the folklore algorithm uses $O(\log d)$ passes.

- *Folding.* Along its popular traces, every limit of the rescaled work-head motion of a linear-time transposer satisfies an isoperimetric inequality that forces the heads to fold back (Theorem M*, [7, Theorems 7.8 and 7.9]). The proof is a cut argument: the information that crosses a cut of the time–space diagram is bounded by the states and the work-tape cells that enter across the cut.

- *Coding.* Every linear-time transposer yields a causal network code for complete exchange on the block network of a time–space diagram of linear length, correct on a $2^{-O(d \log d)}$ fraction of the inputs, and on all inputs if the machine is oblivious [7, Theorems 8.10 and 9.9]. The *Tape Coding Conjecture* says that for large d no causal code solves complete exchange on such a fraction of the inputs on the block network of any diagram of linear length; it implies that (T) fails [7, Conjecture 9.1, Theorem 9.2]. Its plain form, for codes correct on all inputs, excludes oblivious linear-time transposers [7, Theorem 9.9]. On these networks every routing has rate $O(1/\log d)$, so a solving code beats routing by a factor $\Omega(\log d)$, and the conjecture is equivalent to saying that coding beats routing by only $o(\log d)$ on tape networks [7, Theorems 8.8 and 9.4].

The pass bound at polynomial running time, Theorem M* and the reduction to codes survive when the machine is correct on only a $2^{-O(d \log d)}$ fraction of the inputs. The same paper studies an abstract model of the one technique claimed to beat $O(L \log d)$, recursion on XOR-mixed address frames: there every gadget costs at least its number of wires, whereas linear time would need less [7, Theorem 12.4 and Proposition 12.9]; the correspondence between the model and the preprint's recursion is informal. It also records why a proof must use the shape of tapes: a bound $o(\log n)$ on coding gains for bounded-degree graphs in general would give superlinear circuit lower bounds for cyclic shifts and multiplication [7, Proposition 11.2].

The third paper [15] shows that the bit case is at least as hard: linear-time transposition of bit squares of dyadic side implies (T), by one fixed wrapper. The converse is open, and the paper proves barriers for restricted compilers in that direction; one of them is a multiscale obstruction for compilers with boundedly many stages.

3.2 What head motion must do

A second line asks what the heads of a linear-time transposer must do.

- *Not affine.* Along the popular traces, the limits of the rescaled work-head paths of a linear-time transposer fail to be locally affine on sets of times of total measure at least $\frac{1}{2}$, summed over the heads and measured in blocks. Bounded-length words over three explicit palettes of scan operations are excluded, and so are contracting recursions all of whose stages have regular limiting motion [20, Theorems 6.3, 7.1 and 7.5]. No machine whose work heads are home-based

with legs made of boundedly many constant-rate runs transposes in linear time, and in every rescaled limit a linear-time transposer has a linear amount of curved travel [4, Theorems 3.10 and 3.12].

- *Macroscopic crossings.* An oblivious linear-time transposer routes a linear amount of information through pairs of interleaved revisits whose excursions are a fixed fraction of L: for every set of at least αL output bits, every set of steps separating them from the earlier ends of such crossings has at least c times as many elements [12, Theorem 5.5]. A fixed fraction of this expansion sits on one coarsely monotone leg of one tape, in a skeleton from a finite family independent of d, and two work heads must each make linearly many long revisits [12, Theorems 6.7 and 4.1]. The crossing and skeleton statements hold on heavy traces of adaptive machines and of a bit simulator [12, Theorem 8.1].

- *Rows to columns.* In the diagram of an oblivious transposer, every set of steps that separates the inputs of a set of rows from the outputs they feed into a set of output rows carries the corresponding information, whatever the coding [12, Lemma 7.1]. When the cuts for different address bits are disjoint this gives a time bound $\Omega(L \log d)$ that no coding can evade [12, Corollary 7.3].

- *Source covers.* For one fixed geometry of returns, a certificate that retains every input on which an output can depend, and charges the information used to simplify the dependence, gives a correctness deficit of at least εL when the return families consist of a two-class core, at most one arbitrary exceptional family and families of at most εL hops, for fixed bounds on the running time, the capacity and the number of span classes [23, Corollary 4.3]. A memo excludes funnels and single-sweep terminal landings and shows where the certificate stops [5]. These hypotheses on the geometry are not derived for arbitrary machines.

Three results mark the limits of this line.

- *Padding.* For every machine there is a wrapper, with more tapes and states and additive overhead $O(d^2 b)$, whose complete runs satisfy the cut inequality behind Theorem M* with zero slack, and whose normalized head limits satisfy all the conclusions of Theorem M* under a linear time bound. The wrapper's motion is independent of the computation that follows [20, Theorem 8.1]. It needs no idle tapes, and it also meets the finer entry-cell form of the inequality at every slot length from one cell up to a constant number of rows [10]. So these conditions on motion can be met by a machine that transposes nothing; they do not see which information reaches the outputs.

- *One cut for all pairs.* In the diagram of any run, one set of steps separates the inputs from the outputs for every pair of row sets at once [12, Proposition 10.2]. So counting the cuts of the rows-to-columns inequality cannot by itself go beyond linear time: the disjointness that [12, Corollary 7.3] needs is not automatic.

- *The candidates.* Five explicit linear-time cascade schedules meet the crossing and skeleton conditions at the sizes tested. All five are excluded, for fixed work alphabets, by the regular-recursion obstruction of the second paper [21], and two of them also by explicit cuts of the rows-to-columns inequality [12, Theorem 10.8]. Whether the combined conditions on motion decide the oblivious case is open.

3.3 Depth reduction

The fourth paper [4] splits the oblivious case of (T) into two statements.

- *Depth reduction* (the trajectory cut problem TCP(k)): for every $\varepsilon > 0$, deleting εT edges from the dependency graph of k head walks of length T leaves no path with more than a bounded number of long hops, and then, after periodic steps are also revealed, paths of bounded length.
- *The crux*: an index-coding lower bound on the resulting bounded cones, strong enough to exceed the cost of the revealed steps.

Together they would exclude oblivious linear-time transposers [4, Theorem 4.10 and Corollary 4.11]. Depth reduction is now a theorem. It was proved for home-based walks [4, Theorem 4.20]; then for walks of bounded nesting load, and for one arbitrary head together with heads of bounded nesting load at boundedly many scales [2, Theorems 3.6 and 4.2]; and finally for every number of walks, with a depth $D(k, \varepsilon)$ such that $\log \log D = O(k^2 \log(k/\varepsilon))$ [3, Theorem 7.5]. A memo shows that TCP(k) for all k is equivalent to constant-depth revelation of arbitrarily small entropy per input bit, for all linear-time oblivious computations [25]. The fourteenth paper also refutes Conjectures A′ and A″ of the fourth and eighth papers, which would have made the deep-nesting pairs depth-robust [3, Corollary 9.1]: walks nested at m scales are cut to a depth independent of m. It draws the conclusion that matters here [3, Corollary 9.3]:

> *for oblivious machines, the crux alone would exclude linear-time transposers.*

In the language of Section 2.2: for an oblivious linear-time transposer and every $\varepsilon > 0$, revealing εL bits at suitable steps leaves every output bit a function of the revealed word and of boundedly many input bits, and for some value of the revealed word the inputs compatible with it form a binary storage code of rate at least $1 - \varepsilon$ on the machine's own cone relation. The same genies give depth reduction at the level of codes [3, Corollary 9.6], as conjectured in the eleventh paper. And, given the *native half bound* (every binary storage code on the cone relations of the machine's own traces has rate at most $\frac{1}{2} + o(1)$, which is open), they exclude every linear-time transposer, adaptive or not, whose trace has entropy at most ηL for a fixed $\eta < \frac{1}{2}$ [3, Corollary 9.7].

For machines whose data path is linear over a field, linear depth reduction together with a *linear crux*, a min-rank bound for the ball systems of schedules, excludes oblivious linear-time transposers [2, Theorem 7.5]. The linear crux is exactly the relative rigidity of the transposition matrix described in Section 2.3 [2, Problem 10.6], and since the fourteenth paper it alone suffices [3, Corollary 9.3(c)].

3.4 Storage codes: what is proved

The crux needs an upper bound on the rate of storage codes on cone relations. The series proved such bounds, at the pairing rate, for many classes of relations.

- *Homomorphisms and small degree.* Rate bounds pull back along homomorphisms, so every storage code on a graph that maps to C_5, to $K_{14/5}$ or to the Petersen graph has rate at most $\frac{1}{2}$ [4, Theorem 6.5 and Corollary 6.9]. On triangle-free graphs of maximum degree three no scalar linear code over any field beats $\frac{1}{2}$ [4, Theorem 6.13]. The Clebsch graph, of degree five, has guessing number 10 on 16 vertices [4, Corollary 6.11], so bounded degree alone is not enough.
- *Bounded complex coefficients.* On locally bipartite graphs of bounded degree, linear codes with bounded complex coefficients have rate at most $\frac{1}{2} + o(1)$ [4, Theorem 6.17]. The same

holds for complex linear recovery with bounded coefficients and fixed block width on directed relations of bounded incidence degree that are separated, hence on cone relations after a paid trimming of degrees [18, Theorem 4.5 and Corollary 4.7]. On such relations a gain by linear codes with boundedly many symbols per vertex therefore needs positive characteristic, or complex coefficients that no diagonal rescaling makes bounded.

- *Hyperfinite relations.* Every code over every finite alphabet on relations whose incidence graphs are uniformly hyperfinite and separated has rate at most $\frac{1}{2} + o(1)$; no degree bound is needed [18, Corollary 4.4].

- *Nested elimination.* The exact bound $|C|^2 \le |\alpha|^{|V|}$ holds for all codes, linear or not, on large classes of relations without transitive triangles: triangle-free graphs of maximum degree three; loopless relations whose underlying graph has maximum degree three; relations whose vertices of in-degree at least three induce an acyclic sub-relation; relations partitioned into blocks with private outside parents, each block chordless subcubic, a chorded digon path or internally certified; and Cayley relations $\mathrm{Cay}(G, \{s, s^{-1}, t\})$ of every finite group [9, Theorems 4.3, 5.2, 5.3 and 5.10, Corollary 5.12]. It also holds on the class of the fourth paper's Theorem S, for all codes [13, Theorem 3.6]. The degree bound cannot be raised: a 4-regular triangle-free graph on 13 vertices carries a linear code over $\mathbb{F}_4$ with $|C|^2 = 4^{14}$ [9, Example 7.8].

- *Covers and towers.* For the binary adjacency code of the Clebsch graph, every connected tower of binary lifts has kernel dimension at most $N/2 + 2$ [17, Theorem 9.2]; lifting a binary linear code with a unit-diagonal check along a tower of double covers never raises its rate [2, Theorem 8.1].

- *Memos.* The half bound for circular cliques $K_{p/q}$ extends to every $p/q < 3$ [16]; nested elimination extends to soft recovery with record capacities [22]; and the bounds for supports of maximum degree three extend beyond scalar linear codes, to reversible rules over any alphabet and to linear codes whose rows have at most one port of rank above one [24].

One result of this group is about the relations of machines themselves. For cones of fixed depth in linear-time schedules, after a paid trimming of degrees, only $O(L/\log d)$ matched pairs of an input port and the output that demands it lie within any fixed incidence distance [18, Theorem 3.3, Corollaries 3.7 and 3.8]. So cone relations are separated in the sense of Section 2.2, and almost no output is within bounded distance of its own input bit. No gadget of bounded size can then lift a code above the pairing rate: a gain above $\frac{1}{2}$ on boundedly many coordinates needs a short path between the two ports of one of them [18, Lemma 4.1 and Theorem 4.2]. The $L/\log d$ comes from a bound in steps for transport at bounded occupancy, which charges every transported label the $\log d$ bits of its row address [18, Lemma 3.4]: for uncoded movement the logarithm is proved, and what the bound does not cover is a cell that carries a combination of many labels.

This led to the *all-code question* [18, Question 9.1]: does every storage code on separated relations of bounded incidence degree have rate at most $\frac{1}{2} + o(1)$?

3.5 Storage codes: what is false

The answer is no.

- *Above the pairing rate.* There are relations of bounded (astronomically large) incidence degree in which every paired distance tends to infinity, with $\mathbb{F}_{127^2}$-linear storage codes of rate above 0.5021, and binary codes of rate above 0.5013; for large primes inert in $\mathbb{Q}(\sqrt{5})$ the rates approach $4 - 2\sqrt{3} \approx 0.536$ [11, Theorems 7.1 and 7.3, Corollary 8.2]. The relations are twisted quotients

of a Bruhat–Tits tree, the codes are Tanner codes with Reed–Solomon local codes, and their dimension comes from Riemann–Roch on Hilbert modular surfaces in characteristic p. The undirected form of the question, on graphs of bounded degree with long odd cycles, has a negative answer as well [11, Corollary 9.3]. Tanner operators of the same kind, at $p = 3$, refute the independence of the chain in Jaikin-Zapirain's arbitrary-field form of the Lück approximation conjecture [6, Theorem 5.6].

- *Rates approaching one.* A linear-time transposer, even an adaptive one, would yield binary storage codes of this kind at every large degree cap Δ, with rates tending to one as Δ grows [19, Theorem 3.1]; and such codes exist abstractly, with rate at least $1 - C\Delta^{-\alpha}$ [19, Theorem 5.5]. So, at the scale of that transfer theorem, no rate bound for separated relations of bounded degree in general can exclude a transposer [19, Corollary 6.2]: a lower bound must use a further restriction imposed by the physical computation.

No tape process is known that realizes the recoveries of either family.

3.6 Adaptive machines

Much of the above already covers adaptive machines: the pass bound (at polynomial running time, or by the memo), Theorem M* along popular traces, and the reduction of (T) to the Tape Coding Conjecture. Beyond that the series has the following.

- Every linear-time transposer yields an external-memory transposition with $O(d)$ I/Os on blocks of one row that is oblivious on $2^{L-O(d)}$ inputs, so an external-memory lower bound without the indivisibility assumption would refute (T) [8, Theorem 6.4 and Corollary 6.5].
- The source-cover exclusions extend to machines of sufficiently low adaptivity [8, Theorem 6.1].
- Given the native half bound for the relations of skeleton classes, which is open, a linear-time transposer whose work heads enter every cell boundedly often must carry about $L/2$ bits in its coarse skeleton, hence in the coarse motion of its input head [8, Theorem 4.13 and Corollary 4.16]; the fourteenth paper removes the hypothesis on the work heads from the skeleton statement [3, Corollary 9.7].
- The fifteenth paper extracts its codes from any linear-time transposer through a packet compilation [19, Theorem 3.1] and, at the original bit positions, from the traces of a bit simulator with few traces [19, Theorems 8.1 and 9.2]. The crossing theorems hold on heavy traces of the machine and of that simulator [12, Theorem 8.1]; whether they hold in the machine's own diagram on traces that carry almost all inputs is open [12, Problem 11.2].

4 Where the wall is

This section is a diagnosis. It contains no new mathematics. Its claims are consequences of the results cited in Section 3, and where it gives a reading instead of a consequence it says so.

4.1 What is left is one function

For oblivious machines the series has reduced (T) to a question about codes (Section 3.3). Fix the number of tapes, the alphabet and the time constant, and let $\varepsilon > 0$. Depth reduction gives, for every linear-time schedule, a genie that reveals at most εL bits and leaves cones of depth at most

	Paper	Main contribution to the question
1	Tape Coding Conjecture [7]	Formal statement; $\Omega(\log d)$ passes; Theorem M*; transposers are network codes
2	Local Coding Barriers [20]	Local coding bounds; affine and regular motion excluded; padding
3	Bit and Word [15]	Transposition of dyadic bit squares implies (T)
4	Curved Motion [4]	Curved motion; depth reduction plus crux; storage codes at rate $\frac{1}{2}$
5	Directed Recovery [18]	Paired-distance separation of cones; half bounds; the all-code question
6	Covers [17]	Half bounds along covers and for locally generated codes
7	Nested Elimination [9]	Exact all-code half bounds by elimination
8	Bounded Nesting [2]	Depth reduction at bounded nesting; the linear crux as rigidity
9	Source Covers [23]	Source-cover certificates and exclusions
10	Directed Reciprocity [13]	All-code bound on the class of Theorem S; deep nesting
11	Merge Depth [8]	Merge depth; adaptive machines; external memory
12	Lück Approximation [6]	A Tanner operator whose kernel survives along a congruence tower and dies along pro-3 towers
13	Above Half Rate [11]	The all-code question has a negative answer
14	Bounded-Stack DAGs [3]	Depth reduction for every number of walks
15	High-Rate Codes [19]	A transposer needs rates tending to one; such codes exist abstractly
16	Macroscopic Crossings [12]	Crossings, skeletons, rows to columns, one cut for all pairs

Table 1: The earlier papers of the series.

D_ε, a bound that depends on ε and on the fixed parameters but not on d. Write $r(\varepsilon)$ for the best rate, in the limit of large d, of a binary storage code on the cone relations that such genies leave on such schedules. Then:

- an oblivious linear-time transposer needs $r(\varepsilon) \geq 1 - \varepsilon$ for every $\varepsilon > 0$;
- a proof that $r(\varepsilon) < 1 - \varepsilon$ for one ε, for each choice of the fixed parameters, would exclude oblivious linear-time transposers [3, Corollary 9.3];
- the pairing gives $r(\varepsilon) \geq \frac{1}{2}$, and whether $r(\varepsilon) > \frac{1}{2}$ for any ε is open in both directions.

The depth bound matters: without it the cone of an output is everything read before it, and the question is empty. The only general upper bound is the elementary one for relations of degree at most Δ, a rate of $1 - 1/(\Delta + 1)$ [4, Proposition 4.14], which is useless at the depths in question. For machines with linear data paths the same reduction ends in the relative rigidity of the transposition matrix on ball patterns (Section 3.3). For adaptive machines the question arises in two forms: the Tape Coding Conjecture of the first paper, which needs no simulator, and the storage-code criteria on the traces of a bit simulator (Section 3.6).

For particular classes of schedules the exclusion is proved. The sixteenth paper proves it for schedules whose large output sets can be cut cheaply from the macroscopic crossings, and for schedules with boundedly many macroscopic phases [12, Theorems 5.4, 9.2 and 9.3]; the source covers of the ninth paper exclude restricted geometries for all codes. What is missing is a statement for all schedules.

4.2 Three kinds of argument and their limits

Motion without data. The cut inequality behind Theorem M*, its entry-cell refinement and every conclusion of Theorem M* are satisfied by the padded machine of the second paper, which

transposes nothing (Section 3.2). So conditions on head motion that do not refer to the outputs are necessary and cannot be decisive. Conditions that do refer to them go further: the rows-to-columns inequality of the sixteenth paper excludes two of the five candidate schedules. But one cut serves every pair of row sets, so this inequality gives a logarithm only when the cuts for different address bits are disjoint, which is a hypothesis and not a theorem; and what excludes all five candidates is the regular-recursion obstruction, which applies only to schedules all of whose stages have regular limiting motion. Whether conditions of this kind decide the oblivious case is open.

Codes without machines. On abstract separated relations of bounded degree, storage codes exceed the pairing rate, and their rates tend to one polynomially fast as the degree cap grows (Section 3.5). A transposer is only shown to need rates that tend to one very slowly, and the cap grows as ε falls because a genie of cost εL has depth at least of order $1/\varepsilon$ [4, Proposition 4.13]. So no rate bound for separated relations of bounded degree in general can separate a transposer from an abstract code. The two properties of cone relations that the series' coding bounds use, bounded degree after trimming and paired-distance separation, are shared by the counterexamples.

Counting passes. The pass bound of the first paper is the one logarithm proved for a resource of every machine. It saturates one scale at a time: a linear-time run has room for about 2^j legs at scale $2^{-j}L$, while counting at a single scale asks for only $O(j)$ of them, and the threshold of the coarse-pass bound of the sixteenth paper is tight [12, Theorem 10.3 and Corollary 10.4]. That paper states the problem of an inequality in the numbers of legs at all scales at once [12, Problem 11.1]; nothing in the series proves one.

4.3 Why every general tool saturates

The $\log d$ of the folklore algorithm is a sum over $\log_2 d$ address digits. Exchanging one digit of the row address with one digit of the column address costs $O(L)$, and the transposition is the product of these $\log_2 d$ commuting exchanges. Linear time pays for any one of them in full, and for any bounded number.

Every tool of the series *that applies to all machines* is single-scale or scale-free. Cut arguments, macroscopic limits and coarse passes look at one scale at a time, and a linear-time machine has enough steps to satisfy any single-scale requirement. Storage-code bounds are scale-free: they see a relation, not the scale of the addresses that produced it. The one attempt to make a geometric quantity grow with the number of scales failed: walks nested at m scales are cut to a depth independent of m (Section 3.3).

The series does charge scales separately in restricted settings, and they show what the general case lacks.

- For transport at bounded occupancy, each moved label is charged its $\log d$ address bits, in steps [18, Lemma 3.4]. What the bound does not cover is coding.
- For traces whose cuts for different address bits are disjoint, the rows-to-columns inequality gives $\Omega(L \log d)$ against every code [12, Corollary 7.3]. What is not available is the disjointness.
- In the abstract model of XOR-frame recursion every level is charged [7], and compilers with boundedly many stages meet a multiscale obstruction [15]. Both are restricted models.

So the logarithm is proved wherever either the coding or the geometry is tied down, and nowhere else. Corollary 6.13 and Remark 6.14 below make one half of this precise in the linear one-hop

model: every address digit must be treated asymmetrically, but one cheap tape operation does that for all digits at once.

4.4 What the missing statement must be

The limits above describe the statement that would decide the oblivious case. It must

- tie what is known to where it can be, for all schedules: the class results of Section 4.1 do this for restricted geometries;
- fail for abstract separated relations, where codes of rate tending to one exist;
- fail for motion that ignores the data, which padding supplies;
- constrain all scales at once;
- and use the shape of tapes, since a bound on coding gains for bounded-degree graphs in general would be a circuit lower bound (Section 3.1).

Since a linear-time multiplier gives a linear-time transposer [34], a proof of such a statement for all machines would also be the first superlinear lower bound for off-line multiplication on multitape machines.

4.5 What a construction cannot avoid

The same results say what a linear-time machine would have to use.

- *No bounded complex coefficients.* Linear recovery with bounded complex coefficients and bounded width stays at the pairing rate on cone relations (Section 3.4). A linear code would need positive characteristic or coefficients that cannot be rescaled to be bounded; nonlinear codes are not covered.
- *Curved motion.* In the limit, head motion must fail to be locally affine on a positive fraction of the time (Section 3.2), and a recursive schedule must have a stage whose limiting motion is not regular.
- *Expanding relations.* The trimmed incidence graphs of the cone relations must not be uniformly hyperfinite (Section 3.4). This is necessary and cheap: a fixed linear-time machine has such cones and still obeys the all-code half bound [18, Corollary 7.9].
- *No gadgets.* After trimming, almost no output is within bounded distance of its own input bit, so no gain above the pairing rate can come from gadgets of bounded size (Section 3.4). The pairing itself is not excluded; it stops at $\frac{1}{2}$.

Above the pairing rate, at a degree cap that is fixed before the length and the paired distances grow, the only codes known come from one source: sections of line bundles on the supersingular loci of Hilbert modular surfaces [11], by a method that originates in [6], together with their amplifications [19]. At degrees that grow with the length other examples exist.

4.6 The rule and the relation

One feature of machines is absent from the coding statements above. A storage code on a relation lets each receiver choose its own decoding function. A Turing machine has one finite transition table, and on a stretch of time during which its heads move in step, the function that produces the output at one position is the function that produces the output at the next position, translated. The cone relation records which inputs each output can depend on. It does not record that neighbouring outputs depend on them *in the same way*.

Sharing decoders is not by itself an obstruction to rate above one half: by their construction, the linear codes of the thirteenth paper are equivariant under large groups with boundedly many orbits of coordinates. But transposition has a feature that an abstract relation lacks: its demand is not invariant under translating one index. Shifting the columns of a matrix shifts the rows of its transpose. The two theorems of this paper are the two cleanest ways in which structure of the decoder on one index side is incompatible with the swap: *order* (Section 5) and *symmetry* (Section 6). They are proved in the linear one-hop model of Section 2.3. They are not theorems about machines, and Section 7 says what would be needed to make them so.

5 Order: the flag bound

Notation is that of Section 2.3: F is a field, $V = F^d$, τ is the swap, $K = \sum_a A_a \otimes B_a$ with finitely many terms, and the swap code is $\ker(\tau - K) = \{X : X^{\mathsf{T}} = \sum_a A_a X B_a^{\mathsf{T}}\}$. A *complete flag* is a chain of subspaces $W_1 \subset W_2 \subset \cdots \subset W_d = V$ with $\dim W_i = i$.

5.1 The theorem

Theorem 5.1 (Theorem FL, the flag bound)**.** Lean (partial): `finrank_swapCode_le_of_flag`, `finrank_swapCode_le_of_staircase`, `finrank_swapCode_le_of_flag_left`, `finrank_swapCode_swap` [`Flag.lean`]

Not formalized: the choice of bases adapted to the two flags (step 1 of the proof). The Lean statement is the resulting coordinate form: if P, Q are invertible and every $P^{-1}B_aQ$ has its column b' supported on the first $g(b'+1)$ rows, the bound holds.

Let $W_1 \subset \cdots \subset W_d = V$ be a complete flag and put $g(i) = \dim \sum_a B_a(W_i)$. Then, whatever the A_a are,

$$\dim \ker(\tau - K) \;\le\; \sum_{i=1}^{d} g(i).$$

The same holds with the roles of the A_a and the B_a exchanged.

Proof. Step 1: bases. The subspaces $U_i = \sum_a B_a(W_i)$ are nested and $\dim U_i = g(i)$. Choose a basis $f_0, \dots, f_{d-1}$ of V with $W_i = \operatorname{span}(f_0, \dots, f_{i-1})$, and a basis $f'_0, \dots, f'_{d-1}$ with $U_i \subseteq \operatorname{span}(f'_0, \dots, f'_{g(i)-1})$ for every i; such a basis exists because the U_i are nested: extend a basis of U_1 through $U_2, \dots, U_d$ and then to V. For a term $A \otimes B$ of K write $[B]_{b,b'}$ for the coefficient of f'_b in $Bf_{b'}$ and $[A]_{a,a'}$ for the coefficient of f_a in $Af'_{a'}$. Since $Bf_{b'} \in U_{b'+1}$, the coefficient $[B]_{b,b'}$ vanishes unless $b < g(b'+1)$.

Step 2: the equations. Write $X = \sum_{a,b} p_{ab}\, f'_a \otimes f_b$. Then $\tau X = \sum p_{ab}\, f_b \otimes f'_a$, whose coefficient at $f_a \otimes f'_b$ is p_{ba}, and the coefficient of KX at $f_a \otimes f'_b$ is $\sum_{\text{terms}} \sum_{a',b'} [A]_{a,a'} [B]_{b,b'}\, p_{a'b'}$. So $\tau X = KX$ says, for all a, b:

$$p_{ba} \;=\; \text{a linear combination of the } p_{a'b'} \text{ with } g(b'+1) > b. \tag{1}$$

Step 3: elimination. Let $I = \{(a', b') : a' < g(b'+1)\}$, so $|I| = \sum_i g(i)$. We claim that a solution p that vanishes on I is zero; then restriction to I is injective on the code and the bound follows. Suppose p vanishes on I, and prove that row b of p vanishes, by downward induction on $b = d-1, \dots, 0$. By (1), p_{ba} is a combination of entries $p_{a'b'}$ with $g(b'+1) > b$. If $a' > b$, row a' vanishes by induction. If $a' \le b$, then $a' \le b < g(b'+1)$, so $(a', b') \in I$ and $p_{a'b'} = 0$. Hence $p_{ba} = 0$.

Step 4: exchange. Since $\tau(\tau - \sum_a A_a \otimes B_a)\tau = \tau - \sum_a B_a \otimes A_a$, the two kernels have the same dimension. □

Outline of the formal proof of the coordinate form. In Lean the columns are numbered from 0 and the function g is shifted accordingly: Lean's $g(b')$ is the $g(b'+1)$ of the statement above. Let Q be the matrix whose columns are the f_j and P the matrix whose columns are the f'_j; then $(P^{-1}B_aQ)_{b,b'}$ is the coefficient $[B_a]_{b,b'}$. The map $X \mapsto P^{-1}X(Q^{-1})^{\mathsf{T}}$ is an isomorphism from the code of (A_a, B_a) onto the code of $(Q^{-1}A_aP,\ P^{-1}B_aQ)$ (`swapCodeEquiv`, `finrank_swapCode_conj`), which reduces the statement to matrices B_a whose column b' vanishes from row $g(b')$ on. For these, a solution that vanishes on $\{(a', b') : a' < g(b')\}$ is zero, by strong induction on $d-1-b$ (`eq_zero_of_mem_swapCode_of_staircase`), so restriction to that set is injective (`staircaseRestrict_injective`), and the set has $\sum_{b'} \min(g(b'), d)$ elements (`card_staircase`). The exchange of sides is `finrank_swapCode_swap`. No monotonicity of g is used. □

The bound depends on one tensor side only, and on that side only through how fast the images of a flag grow: we call g the *expansion profile* of the B_a along the flag. The transposed profiles are also available, because $\operatorname{rank}(\tau - K) = \operatorname{rank}(\tau - K^{\mathsf{T}})$ and $K^{\mathsf{T}} = \sum_a A_a^{\mathsf{T}} \otimes B_a^{\mathsf{T}}$.

Proposition 5.2 (The bound is attained for every profile)**.** Lean: `finrank_swapCode_staircase`, `exists_staircase_finrank_eq`, `mem_swapCode_staircase_iff` [`Flag.lean`]

Let $g\colon \{1, \dots, d\} \to \{0, \dots, d\}$ be any function. Take one term for every pair (i, k) with $k < g(i+1)$, namely $A = E_{ik}$ and $B = E_{ki}$, where E_{ik} is the matrix unit. Then column i of every B_a is supported on the first $g(i+1)$ rows, and the swap code is $\{X : X_{ki} = 0$ unless $k < g(i+1)\}$, of dimension exactly $\sum_i g(i)$.

Outline of the formal proof. Here $KX = \sum X_{ki}E_{ik}$ over the chosen pairs, so $X^{\mathsf{T}} = KX$ holds at an entry (i, k) with $k < g(i+1)$ and forces $X_{ki} = 0$ at the others (`mem_swapCode_staircase_iff`). □

Remark 5.3 (Numerical: the bound is not always tight)**.** For a fixed span of second factors the best flag need not give the true maximum over the first factors. In an exhaustive computation over $\mathbb{F}_2$ at $d = 3$, for 4 of the 20 orbits of spans of at most two second factors the maximum is 4 while the best flag, over $\mathbb{F}_2$, $\mathbb{F}_4$ or $\mathbb{F}_8$, gives 5 or 6.

5.2 Corollaries

Write $h = d(d-1)/2$, so that $\dim\ker(\tau - K) \le d(d+1)/2$ is the same as $\operatorname{rank}(\tau - K) \ge h$.

Corollary 5.4 (A pair of flags)**.** Lean (partial): `two_mul_finrank_swapCode_le_of_triangular`, `two_mul_finrank_swapCode_le_of_triangular_left` [`Flag.lean`]

Not formalized: the passage from a pair of flags to bases in which the B_a are upper triangular, as in step 1 of the proof of Theorem 5.1. The Lean statement is the matrix form.

If $P^{-1}B_aQ$ is upper triangular for all a, for some invertible P and Q, then $\dim\ker(\tau - K) \le d(d+1)/2$, for arbitrary A_a. In terms of flags: this holds if there are complete flags W and W' with $B_a(W_i) \subseteq W'_i$ for all a and i.

Outline of the formal proof. This is Theorem 5.1 with $g(i) \le i$, and $\sum_{i \le d} i = d(d+1)/2$ (`two_mul_sum_fin_succ`). Bases adapted to W and W' make the B_a upper triangular, and conversely. □

So the pairing value is the value of a hidden order: a pair of flags is an order in which the t-th generalized column is computed from the first t generalized rows.

Corollary 5.5 (At most two terms, every field)**.** Not formalized; a complete proof is given below.
For all $A_1, A_2, B_1, B_2 \in \mathrm{M}_d(F)$,

$$\dim\{X : X^{\mathsf{T}} = A_1 X B_1^{\mathsf{T}} + A_2 X B_2^{\mathsf{T}}\} \;\le\; \frac{d(d+1)}{2}.$$

Proof (not formalized). The dimension of a kernel does not change under extension of the field, so let F be algebraically closed. We show that any two linear maps $B_1, B_2 \colon V \to V'$ between spaces of the same dimension d admit complete flags with $B_j(W_i) \subseteq W'_i$; Corollary 5.4 then applies. The determinant $\det(\alpha B_1 + \beta B_2)$, in any bases, is a binary form of degree d, so it vanishes at some $(\alpha, \beta) \neq 0$ (at every (α, β) if it is identically zero). Take $v \neq 0$ with $(\alpha B_1 + \beta B_2)v = 0$. Then $B_1 v$ and $B_2 v$ are proportional; let $W_1 = \langle v \rangle$ and let W'_1 be a line containing both. The maps induced on $V/W_1 \to V'/W'_1$ are again two maps between spaces of equal dimension, and induction on d gives the flags. □

Remark 5.6 (Two is sharp)**.** Three terms can exceed the pairing value. For every d, the operator $\Pi = \sum_{i,j} E_{ij} \otimes E_{ij}$ is $X \mapsto \mathrm{tr}(X) \cdot 1$, of rank one, and

$$\tau - \Pi \;=\; \sum_{i<j} (E_{ij} - E_{ji}) \otimes (E_{ji} - E_{ij})$$

has h terms. So $K = \tau - \Pi$ has h terms and its code $\ker \Pi$, the matrices of trace zero, has dimension $d^2 - 1$, in every characteristic. At $d = 3$ that is three terms and dimension $8 > 6$. At $d = 4$, adding E_{33} to both factors of the third term of the $d = 3$ example, with indices $0, \ldots, 3$, gives three terms and a code of dimension $11 > 10$ (numerical: checked over $\mathbb{F}_2, \mathbb{F}_3, \mathbb{F}_5, \mathbb{F}_7$ by `tools/three_term_example.py`). We have no three-term example for $d \ge 5$.

Corollary 5.7 (One triangularizable side)**.** Not formalized; a complete proof is given below.
Suppose that, over the algebraic closure of F, the B_a have a common invariant complete flag. Then $\dim \ker(\tau - K) \le d(d+1)/2$ for any number of terms and arbitrary A_a. This holds in particular when

(a) the B_a commute pairwise, for example when all are circulant;

(b) $\mathrm{char}\, F = p$ and all B_a lie in the linear span of one finite p-group of matrices.

Proof (not formalized). An invariant flag gives Corollary 5.4 with $W' = W$, over the closure, and the dimension does not depend on the field. (a) Commuting matrices over an algebraically closed field have a common eigenvector: an eigenspace of one of them is invariant under the others, and one inducts on the dimension. Passing to the quotient by that eigenvector and inducting again gives a common invariant flag. (b) Let G be a finite p-group of invertible matrices in characteristic p and $z \neq 1$ a central element of order p. Then $(z-1)^p = z^p - 1 = 0$, so $\ker(z-1)$ is a nonzero G-invariant subspace on which $G/\langle z \rangle$ acts; by induction on the order of the group, G fixes a nonzero vector. Passing to the quotient by that vector and inducting on the dimension gives a complete flag invariant under G, hence under its span. □

For circulant B_a the statement reads: *if every column of X is a linear function of the rows of X that commutes with cyclic rotation of the rows' coordinates (any number of rows, each rotated and convolved with kernels that may depend on the column), then X ranges over a space of dimension at most $d(d+1)/2$.* Reading any number of source rows in step with the output cannot beat the symmetric matrices.

In (b) the group must be the same for all B_a. For $p = 2$ and $d = 2^m$ two natural examples are the automorphisms of the hierarchy of dyadic intervals (block swaps, block reversals, XOR translations) and the affine group $\{i \mapsto ui + t : u \text{ odd}\}$ of $\mathbb{Z}/2^m$. They are different subgroups of the symmetric group, and mixing them is not covered: at $d = 4$ a rotation and the transposition (0 1), which swaps the halves of one dyadic block of length two, generate the whole symmetric group. The next corollary covers more of the second example.

Corollary 5.8 (All affine index maps in characteristic p)**.** Not formalized; a complete proof is given below.

Let $\operatorname{char} F = p$ *and* $d = p^m$*. For* $u, t \in \mathbb{Z}/d$ *let* $\varphi_{u,t}$ *be the linear map on* $F^{\mathbb{Z}/d}$ *with* $(\varphi_{u,t}x)_i = x_{ui+t}$*; here* u *need not be a unit. If every* B_a *lies in the span of the* $\varphi_{u,t}$*, then* $\dim\ker(\tau - K) \le d(d+1)/2$.

Proof (not formalized). Let $R = \varphi_{1,1}$ be the rotation. Then $R\varphi_{u,t} = \varphi_{u,t}R^u$, since both send x to $(x_{ui+t+u})_i$. Hence $(R-1)^i\varphi_{u,t} = \varphi_{u,t}(R^u - 1)^i$, and $R^u - 1$ is a multiple of $R - 1$ in the commutative algebra $F[R]$, so $\varphi_{u,t}$ maps $\ker(R-1)^i$ into itself. As R is a single cycle of length p^m in characteristic p, $R - 1$ is nilpotent with a one-dimensional kernel, and the subspaces $\ker(R-1)^i$, $1 \le i \le d$, form a complete flag. Apply Corollary 5.4 with $W' = W$. □

So over $\mathbb{F}_2$, on rows of length 2^m, rotations, reflections, dilations, wrap-around decimations and convolutions all preserve one flag, in any combination and at any depth (Remark 5.11). What leaves that world is a projection: a mask that selects part of a row.

Corollary 5.9 (Blocks and expansion)**.** Not formalized; a complete proof is given below.

(a) If, for suitable invertible P, Q, all $P^{-1}B_aQ$ are block upper triangular with diagonal blocks of sizes $d_1, \dots, d_m$, then $\dim\ker(\tau - K) \le (d^2 + \sum_s d_s^2)/2$.

(b) If some flag has $g(i) \le ci + c_0$ with $c \ge 1$ and $c_0 \ge 0$, then $\dim\ker(\tau - K) \le (1 - \frac{1}{2c})d^2 + (\frac{c_0}{c} + \frac{1}{2})d$.

(c) A sum of k terms has $\dim\ker(\tau - K) \le \sum_{i \le d} \min(d, ki) \le (1 - \frac{1}{2k})d^2 + \frac{d}{2}$.

(d) Let d be prime to $\operatorname{char} F$ *and $s \ge 1$. If every B_a lies in the span of the maps $\varphi_{u,t}$ of Corollary 5.8 with $u \in \{-s, \dots, s\}$, then* $\dim\ker(\tau - K) \le (1 - \frac{1}{2s})d^2 + (\frac{1}{s} + \frac{1}{2})d$.

Proof (not formalized). (a) Take $g(i) = d_1 + \cdots + d_s$ for i in block s; then $\sum_i g(i) = \sum_s d_s(d_1 + \cdots + d_s) = (d^2 + \sum_s d_s^2)/2$. (b) If $d < c_0$ the right side exceeds d^2. Otherwise put $u = (d - c_0)/c$. Then $\sum_i \min(d, ci + c_0) = d^2 - \sum_{i \le u} c(u - i) \le d^2 - \frac{c}{2}(u^2 - u)$, because $\sum_{i \le \lfloor u \rfloor}(u - i) \ge (u^2 - u)/2$, and $d^2 - \frac{(d-c_0)^2}{2c} + \frac{d-c_0}{2}$ is at most the claimed bound. (c) Any flag has $g(i) \le ki$; apply (b). (d) Extend the field so that it contains a primitive d-th root of unity, and let χ_k be the characters of $\mathbb{Z}/d$. Then $\varphi_{u,t}\chi_k$ is a multiple of χ_{uk}. Order the frequencies $0, 1, -1, 2, -2, \dots$ and let W_i be spanned by the first i characters. The frequencies uk with $|u| \le s$ and $\chi_k \in W_i$ lie in an interval of at most $si + 1$ residues, so $g(i) \le si + 1$; apply (b). □

The hypothesis $c \ge 1$ in (b) is needed: for $B = \operatorname{diag}(1_{d/2}, 0)$, $A = 1$ and a flag that alternates kernel and non-kernel vectors, $g(i) = \lfloor i/2 \rfloor$ and the code has dimension $d(d+2)/8$.

Remark 5.10 (Entries of b bits)**.** For $K = \sum_a \tilde{A}_a \otimes B_a$ on $V \otimes V \otimes F^b$ with $\tilde{A}_a \in \mathrm{End}(V \otimes F^b)$ arbitrary and $B_a \in \mathrm{End}(V)$, the proof of Theorem 5.1 gives $\dim\ker(\tau \otimes 1 - K) \le b \sum_i g(i)$, with coefficients $p_{ab} \in F^b$. Operators that mix the column index with the position inside an entry are covered as well: every operator on $V \otimes F^b$ is a sum $\sum_j B_j \otimes C_j$, the C_j can be absorbed into the first factors, and the profile is that of the B_j. For a cyclic shift of the bit string of a row by a number of bits that is not a multiple of b, the B_j are two rotations.

Remark 5.11 (Composition)**.** If the second factors of $K_1, \dots, K_t$ all lie in one algebra $\mathfrak{B} \subseteq \mathrm{M}_d(F)$, so do those of the product $K_t \cdots K_1$, because $(A \otimes B)(A' \otimes B') = AA' \otimes BB'$. So Corollaries 5.7 and 5.8 are statements about linear data paths of any depth: a path that acts on one index through a commutative algebra, or through the affine maps of Corollary 5.8, and on the other index in any way at all, is correct on at most the pairing value.

5.3 What the theorem says

- *Only the weaker side counts.* The bound uses the expansion of the A_a or of the B_a, whichever is smaller. Arbitrary mixing on one index buys nothing while the other index has a flag.
- *A code above the pairing value needs expansion on both sides*: for every flag on either side, the images of the flag must grow faster than the flag. By Corollary 5.9(b), a profile $g(i) \le ci$ with $c \ge 1$ gives rate at most $1 - 1/(2c) + O(1/d)$, so rate $1 - \varepsilon$ needs expansion factor about $1/(2\varepsilon)$ along every flag, on both sides.
- *Expansion is cheap on a tape.* Over the algebraic closure, rotations of a row have an invariant flag, and so do masks by intervals, but the two flags are incompatible: rotations together with the mask of one proper interval generate the full matrix algebra. For rotations with the masks of M intervals J_l, a rotation-invariant flag has $g(i) = \sum_l \min(i, |J_l|) \le Mi$, and no flag does better, because the rotations of W_i span a rotation-invariant space of dimension at least i, and such a space of dimension h restricts to an interval of length m in dimension $\min(h, m)$. For M equal intervals this gives rate $1 - 1/(2M)$ and no more. So the flag bound cannot hold that family to the pairing rate. The next section does.

6 Symmetry: the translate pigeonhole

The flag bound cannot hold rotations with interval masks to the pairing rate (Section 5.3). This section does, by a different argument: a code larger than half the space meets its own translate, and on the intersection a shared decoder forces the data to be translation-covariant.

6.1 The lemma

For operators S, T write $[S, T] = ST - TS$.

Lemma 6.1 (Pigeonhole against a translate)**.** Lean: `two_mul_finrank_ker_sub_le_ker`, `finrank_ker_commutator_sub_le`, `two_mul_finrank_ker_sub_le`, `two_mul_finrank_ker_sub_le_of_commute`, `finrank_range_commutator_le` [`Pigeonhole.lean`]
Let τ, K, T be linear operators on a space of finite dimension N over a field. Then

$$\dim\ker(\tau - K) \;\le\; \tfrac{1}{2}(N + \dim\ker[\tau - K, T]) \;\le\; \tfrac{1}{2}(N + \dim\ker[\tau, T] + \mathrm{rank}[K, T]).$$

Proof. Let $\mathcal{X} = \ker(\tau - K)$, $D = \dim \mathcal{X}$ and $\mathcal{X}_T = \{X \in \mathcal{X} : TX \in \mathcal{X}\}$. The map $\mathcal{X} \to F^N/\mathcal{X}$, $X \mapsto TX + \mathcal{X}$, has kernel $\mathcal{X}_T$ and a target of dimension $N - D$, so $\dim \mathcal{X}_T \geq 2D - N$; T need not be invertible. For $X \in \mathcal{X}_T$, $(\tau - K)TX = 0 = T(\tau - K)X$, so $\mathcal{X}_T \subseteq \ker[\tau - K, T]$. That is the first inequality. For the second, $[\tau - K, T]X = 0$ means $[\tau, T]X = [K, T]X \in \operatorname{im}[K, T]$, and the preimage of a subspace W under a map has dimension at most that of its kernel plus $\dim W$. □

With $S = \tau - K$ the first inequality says $\operatorname{rank}[S, T] \leq 2 \operatorname{rank} S$: a commutator has rank at most twice the rank of either factor. The Lean proof takes this route, from $\operatorname{rank}[S, T] \leq \operatorname{rank}(ST) + \operatorname{rank}(TS)$ and rank–nullity (`finrank_range_commutator_le`). The lemma is elementary, and we claim no novelty for it. What is used below is the form with τ and K separated: K may nearly commute with a T with which τ does not.

Proposition 6.2 (For the swap, the best case is the pairing value)**.** Lean (partial): `card_le_finrank_ker_swap_commutator`, `two_mul_finrank_ker_swap_sub_one`, `two_mul_finrank_range_swap_sub_one` [`Swap.lean`]
Not formalized: the equality case for $T = 1 \otimes G$.

On $V \otimes V$ with $\dim V = d$, every operator T has $\dim \ker[\tau, T] \geq d$, in every characteristic. Equality holds for $T = 1 \otimes G$, that is $X \mapsto XG^{\mathsf{T}}$, when G is non-derogatory.

Proof. The kernel of $\tau - 1$ is the space of symmetric matrices, of dimension $d(d+1)/2$ in every characteristic, so $\operatorname{rank}(\tau - 1) = d(d-1)/2$, and $[\tau, T] = [\tau - 1, T]$ has rank at most $d(d-1)$ by Lemma 6.1. For $T = 1 \otimes G$, $[\tau, T]X = GX^{\mathsf{T}} - X^{\mathsf{T}}G^{\mathsf{T}}$, so the kernel is the space of $Y = X^{\mathsf{T}}$ with $GY = YG^{\mathsf{T}}$. As G^{T} is similar to G, this space is isomorphic to the centralizer of G, whose dimension is d exactly when G is non-derogatory. □

So Lemma 6.1 never gives less than $(d^2 + d)/2$ for the swap, and it gives exactly the pairing value when K commutes with a suitable T. The rest of the section is about K that commute with a translation up to a boundary.

6.2 Class-uniform rules

Let σ be the cyclic rotation of $F^{\mathbb{Z}/d}$, $(\sigma x)_j = x_{j+1}$. For a matrix X with rows $x_c \in F^{\mathbb{Z}/d}$, $c \in \mathbb{Z}/d$, write $\sigma^\delta X$ for the matrix with rows $\sigma^\delta x_c$.

Definition 6.3 (Class-uniform rules)**.** Lean: `rot`, `IsClassUniformCode` [`ClassUniform.lean`]
Fix a partition of the row index $\mathbb{Z}/d$ into non-empty classes. A *class-uniform rule* is a family of linear maps $\mathcal{A}_J \colon \mathrm{M}_d(F) \to F^{\mathbb{Z}/d}$, one for each class J. A subspace $\mathcal{X} \subseteq \mathrm{M}_d(F)$ is a *code* of the rule if

$$x_c = \mathcal{A}_J(\sigma^c X) \qquad \text{for every } X \in \mathcal{X}, \text{ every class } J \text{ and every } c \in J.$$

In words: row c of X is a fixed linear function of the whole matrix with its columns rotated by c, and the function depends only on the class of c. The receiver of row c looks at the stored word from its own position, through a window of any size, and receivers in one class use the same decoder.

In the notation of Section 2.3 (this translation is not formalized): let $K = \sum_a A_a \otimes B_a$ have second factors in the span of the matrices $P_J\sigma^e$, a rotation followed by the mask P_J of a class, and arbitrary first factors, say $K = \sum_{J,e} A_{J,e} \otimes P_J\sigma^e$. Then $(KX)_{r,c} = \sum_{i,e}(A_{J,e})_{r,i}X_{i,c+e}$ for $c \in J$, so the swap code of K is the largest code of the rule $\mathcal{A}_J(Y)_r = \sum_{i,e}(A_{J,e})_{r,i}Y_{i,e}$, and every rule arises in this way. This is the family of Section 5.3. For a single class the B_a are circulant and Corollary 5.7 applies. If the mask is applied before the rotation, K^{T} has second factors $P_J\sigma^{-e}$, and since $\operatorname{rank}(\tau - K) = \operatorname{rank}(\tau - K^{\mathsf{T}})$ the bounds below hold for that family as well.

Theorem 6.4 (Theorem TP, the translate pigeonhole)**.** Lean: `IsClassUniformCode.finrank_le`, `Link`, `IsClassUniformCode.two_mul_finrank_le`, `IsClassUniformCode.two_mul_finrank_le_of_const` [`ClassUniform.lean`]

Let $\mathcal{X}$ be a code of a class-uniform rule. Let $\Delta_0 \subseteq \mathbb{Z}/d$ be a set of $s+1 \geq 2$ shifts, and let χ be the number of connected components of the graph on $\mathbb{Z}/d$ in which $c \sim c'$ when c and c' lie in one class and $c' - c \in \Delta_0 - \Delta_0$. Then

$$\dim \mathcal{X} \;\leq\; \frac{s\,d^2 + d\,\chi}{s+1}.$$

In particular, if the classes are M cyclic intervals, then $\dim \mathcal{X} \leq (d^2 + Md)/2$.

Outline of the formal proof. Let $\mathcal{X}' = \{X : \sigma^\delta X \in \mathcal{X} \text{ for all } \delta \in \Delta_0\}$, an intersection of $s+1$ preimages of $\mathcal{X}$; each has codimension at most $d^2 - \dim \mathcal{X}$, so $\dim \mathcal{X}' \geq (s+1)\dim \mathcal{X} - s\,d^2$ (`card_mul_finrank_add_le_iInf_comap`, `mul_finrank_le_add_finrank_iInf_comap`, in `Pigeonhole.lean`). Let $X \in \mathcal{X}'$, $\delta, \delta' \in \Delta_0$ and c, c' in one class J with $c + \delta = c' + \delta'$. The rule applied to $\sigma^\delta X$ at row c gives $\sigma^\delta x_c = \mathcal{A}_J(\sigma^{c+\delta} X)$, and applied to $\sigma^{\delta'} X$ at row c' it gives $\sigma^{\delta'} x_{c'} = \mathcal{A}_J(\sigma^{c'+\delta'} X)$. The right sides agree, so $x_{c'} = \sigma^{c'-c} x_c$ (`IsClassUniformCode.row_eq_of_rot_mem`). Hence each row of X determines the rows in its component, restriction to one row per component is injective on $\mathcal{X}'$, and $\dim \mathcal{X}' \leq d\chi$. The Lean statement takes any set S of rows that meets every component and gives $(s+1)\dim \mathcal{X} \leq s\,d^2 + d|S|$. For M cyclic intervals and $\Delta_0 = \{0, 1\}$ the components are the intervals. In Lean this case (`IsClassUniformCode.two_mul_finrank_le`) takes a non-empty set S containing every row whose class differs from that of the row before it, so that $|S| = M$ for $M \geq 2$ intervals and S is any single row for one class (`IsClassUniformCode.two_mul_finrank_le_of_const`). □

The space on which the proof lands is worth naming. For one unit shift it is $\mathcal{H} = \{X : x_{c+1} = \sigma x_c \text{ whenever } c, c+1 \text{ lie in one class}\}$: inside each interval the matrix is constant along anti-diagonals. It has d free parameters per interval, whatever the length of the interval.

We say that a partition into M classes has an *almost-period* if some single shift δ leaves only M components in the graph of Theorem 6.4 with $\Delta_0 = \{0, \delta\}$, so that the bound is $(d^2 + Md)/2$. Intervals have the almost-period 1, and residue classes modulo M have the almost-period M.

Lemma 6.5 (Extremal codes contain $\mathcal{H}$)**.** Not formalized; a complete proof is given below.

Let the classes be M cyclic intervals and let $\mathcal{X}$ be a code of a class-uniform rule with $2\dim \mathcal{X} = d^2 + Md$. Then $\mathcal{X} \cap \sigma^{-1}\mathcal{X} = \mathcal{H}$; in particular $\mathcal{H} \subseteq \mathcal{X}$.

Proof (not formalized). By the proof of Theorem 6.4, $\mathcal{X}' = \mathcal{X} \cap \sigma^{-1}\mathcal{X}$ lies in $\mathcal{H}$ and has dimension at least $2\dim \mathcal{X} - d^2 = Md = \dim \mathcal{H}$. □

Proposition 6.6 (Equal pieces: the bound is attained)**.** Lean: `blockSymmetric`, `isClassUniformCode_blockSymmetric`, `two_mul_finrank_blockSymmetric`, `blockSymmetric_attains_bound` [`BlockSymmetric.lean`]

Let $d = Mm$ with $M, m \geq 1$ and let the classes be the M intervals of length m. The block-symmetric *matrices, those whose $m \times m$ blocks in the $M \times M$ grid are all symmetric, are a code of a class-uniform rule, of dimension $(d^2 + Md)/2$.*

Outline of the formal proof. Write a row index as $c = sm + k$ and a column index as $r = tm + l$ with $0 \leq k, l < m$. Block symmetry says $X_{sm+k,\,tm+l} = X_{sm+l,\,tm+k}$. The right side is the entry of $\sigma^c X$ in row $sm + l$ and column $tm + k - c = (t - s)m$, which depends on c only through its block s. So $x_c = \mathcal{A}_s(\sigma^c X)$ with $\mathcal{A}_s(Y)_{tm+l} = Y_{sm+l,\,(t-s)m}$ (`isClassUniformCode_blockSymmetric`). There are M^2 blocks, each with $m(m+1)/2$ free entries, and $M^2 m(m+1)/2 = (d^2 + Md)/2$ (`two_mul_finrank_blockSymmetric`); the set of the M first rows of the blocks satisfies the hypothesis of Theorem 6.4 for the unit shift (`blockSymmetric_attains_bound`). □

The operator K of this rule is the coarse transposition of the grid: $X \mapsto (\mathcal{A}_{s(c)}(\sigma^c X))_c$, read as a matrix with these columns, has the entry $X_{sm+l,\,tm+k}$ in position $(tm+l,\ sm+k)$, so block (s,t) is moved to position (t,s) with its inner coordinates kept.

So the largest code of a class-uniform rule with M equal interval pieces has dimension exactly $(d^2+Md)/2$, the value of the $M \times M$ coarse transposition, against the bound $1-1/(2M)$ that flags give for the same family. For $M=d$ the pieces are single rows, the rule is unrestricted and the code is everything. In between, each additional piece raises the bound by $d/2$.

Remark 6.7 (Numerical: what the exact value does not say)**.** Exhaustive computations over $\mathbb{F}_2$ show the following. (a) Extremal codes are far from unique: for $d=4$ and classes $\{0,1\},\{2,3\}$ there are 59 392 codes of dimension 12, in 327 orbits under the evident symmetries, and the block-symmetric code is one orbit. (b) Coding beats copying inside the bound: for $d=4$ and pieces $\{0\},\{1,2,3\}$ the bound 12 is attained, but no rule that copies one stored entry to each output entry exceeds 11. (c) The bound is not attained for every partition into unequal intervals: by Lemma 6.5 and a solver, the pieces $\{0\},\{1\},\{2,3,4\}$ at $d=5$ have largest code $19<20$ (Remark 8.1).

Proposition 6.8 (Copy rules, every partition)**.** Not formalized; a complete proof is given below.

Call a class-uniform rule a copy rule *if every output entry is a multiple of one stored entry:* $X_{c,r} = \lambda_{J,r}\, X_{\pi_J(r),\,\rho_J(r)+c}$ *for* $c \in J$*. A code of a copy rule with* M *classes, of any shape, has dimension at most* $(d^2+Md)/2$*.*

Proof (not formalized). Let $f(c,r) = (\pi_J(r), \rho_J(r)+c)$ for $c \in J$, a map on the d^2 positions. A matrix in the code is determined, on each component of the functional graph of f, by its value at one position on the cycle of that component, so the dimension is at most the number of cycles. A fixed point (c,r) has $\pi_J(r)=c$ with J the class of c, so for each r there is at most one fixed point per class: at most Md in all. Every other cycle has at least two positions. □

So for copy rules the shape of the classes does not matter, while for coded rules Theorem 6.4 needs the classes to have an almost-period. The difference is real: Proposition 8.2 exhibits two classes with no almost-period and a coded rule whose code has dimension greater than $(d^2+Md)/2$. How far coded rules on such classes can go is Problem 9.1.

6.3 On a line

A tape is a line, not a cycle. Let rows and columns be indexed by $\{0,\dots,d-1\}$ and let the classes be intervals. A *two-sided rule on a line* is given by coefficients $\mathcal{A}_{J,i}[r][e]$, $-d<e<d$, with

$$X_{c,r} = \sum_i \sum_{j=0}^{d-1} \mathcal{A}_{J,i}[r][j-c]\, X_{i,j} \qquad (c \in J).$$

The decoder of row c again sees the stored word from position c, through a window that is cut off at the ends instead of wrapping around. Every class-uniform rule is of this form, with the same classes: take $\mathcal{A}_{J,i}[r][e]$ to depend on e modulo d (this remark is not formalized). On a line, an interval is a set of consecutive rows that does not wrap around.

Theorem 6.9 (The line form of Theorem TP)**.** Lean: `IsLineUniformCode`, `IsLineUniformCode.two_mul_finrank_le`, `isLineUniformCode_of_coeff`, `lineView`, `truncShift` [`Line.lean`]

A code of a two-sided rule on a line with M *interval classes has dimension at most* $(d^2+Md)/2$*.*

Outline of the formal proof. Let $(TX)_{i,j} = X_{i,j+1}$ for $j < d-1$ and $(TX)_{i,d-1} = 0$, and $\mathcal{X}' = \{X \in \mathcal{X} : TX \in \mathcal{X}\}$, so that $\dim \mathcal{X}' \geq 2 \dim \mathcal{X} - d^2$. Let $X \in \mathcal{X}'$ and let $c, c+1$ lie in one class J. The rule for TX at row c reads $X_{c,r+1} = \sum_i \sum_{j \geq 1} \mathcal{A}_{J,i}[r][j-(c+1)] X_{i,j}$, with $X_{c,d}$ read as 0, and the rule for X at row $c+1$ has the same sum with $j \geq 0$. So

$$X_{c+1,r} - X_{c,r+1} = \sum_i \mathcal{A}_{J,i}[r][-(c+1)]\, X_{i,0}.$$

Suppose that column 0 of X vanishes. Then inside each class, with rows $a, \ldots, a + m_J - 1$, the matrix is constant along anti-diagonals. An anti-diagonal that meets column 0 inside the class is zero, and so is one whose first position in the class is in the last column below row a, because $X_{c,d} = 0$. The remaining anti-diagonals start in row a at a column $r \geq m_J$: there are $d - m_J$ of them. So $\dim \mathcal{X}' \leq d + \sum_J (d - m_J) = Md$. In Lean the decoder of a class is an arbitrary linear function of the zero-padded window $(X_{i,e+c})_{i,e}$ (`lineView`); the coefficient form above is an instance (`isLineUniformCode_of_coeff`). The classes need not be intervals there: the bound is $(d^2 + d|S|)/2$ for any set S of rows that contains row 0 and every row whose class differs from that of the row before it. The key step is `IsLineUniformCode.row_succ_eq`. □

With one class this gives the pairing value for second factors in the span of the truncated shifts, the Toeplitz matrices, which do not commute and are not covered by Corollary 5.7; the symmetric matrices attain it (`isLineUniformCode_symmetric`). For M equal intervals the block-symmetric matrices attain the bound on a line as well, by the remark above; this is not formalized.

Remark 6.10 (Numerical: on a line, other shifts fail)**.** For $d = 3$ and classes $\{0, 2\}, \{1\}$, Theorem 6.4 with $\Delta_0 = \{0, 2\}$ gives 7 on a cycle. On a line the largest code of a two-sided rule has dimension 8 (exhaustive over $\mathbb{F}_2$); one such code is $\{X : X_{0,0} = X_{2,2}\}$. So Theorem 6.9 does not extend to shift sets other than $\{0, 1\}$.

6.4 Twists, digits and composition

A rule need not commute with a translation to be caught; it is enough that it intertwines it with boundedly many others.

Theorem 6.11 (Twist classes)**.** `Lean: succ_mul_finrank_swapCode_le_of_twist_classes, two_mul_finrank_swapCode_le_of_twist, twistSpace, succ_mul_finrank_swapCode_le, finrank_le_sum_finrank_twistSpace` [`Twist.lean`]

Let $G \in \mathrm{M}_d(F)$, let the terms of $K = \sum_a A_a \otimes B_a$ fall into classes $1, \ldots, s$, and let $G'_1, \ldots, G'_s$ commute pairwise, with $GB_a^{\mathsf{T}} = B_a^{\mathsf{T}} G'_\kappa$ for every a in class κ. Then

$$\dim \ker(\tau - K) \;\leq\; \frac{s\, d^2 + \sum_\kappa h_\kappa}{s+1}, \qquad h_\kappa = \dim\{P : G^{\mathsf{T}} P = P G'_\kappa\}.$$

Outline of the formal proof. Let $\mathcal{X}' = \{X : XG^t \in \ker(\tau - K),\ 0 \leq t \leq s\}$, of dimension at least $(s+1)\dim\ker(\tau - K) - s\, d^2$ (`succ_mul_finrank_swapCode_le`). For $X \in \mathcal{X}'$ and $P = X^{\mathsf{T}}$ the hypothesis gives $(G^{\mathsf{T}})^t P = \sum_\kappa T_\kappa G'^t_\kappa$ for $0 \leq t \leq s$, with $T_\kappa = \sum_{a \in \kappa} A_a X B_a^{\mathsf{T}}$. Applying $P \mapsto G^{\mathsf{T}} P - P G'_\kappa$ removes class κ from such a system and lowers s by one, and its kernel is the space counted by h_κ; induction on the number of classes gives $\dim \mathcal{X}' \leq \sum_\kappa h_\kappa$ (`finrank_le_sum_finrank_twistSpace`). This is the statement that $\prod_\kappa (L - R_\kappa) P = 0$ for the commuting operators L, left multiplication by G^{T}, and R_κ, right multiplication by G'_κ. □

Corollary 6.12 (Modulation frequencies)**.** Not formalized; a complete proof is given below.

Let F contain a primitive d-th root of unity ω and let $Z = \operatorname{diag}(1, \omega, \ldots, \omega^{d-1})$. If every B_a^{T} is a product $Z^k C$ with C circulant and k in a fixed set of s frequencies, then $\dim\ker(\tau - K) \le \frac{s}{s+1}(d^2+d)$.

Proof (not formalized). $\sigma Z = \omega Z \sigma$, so $\sigma Z^k C = Z^k C\,(\omega^k \sigma)$: take $G = \sigma$ and $G'_k = \omega^k \sigma$, which commute. The matrices $\sigma^{\mathsf{T}} = \sigma^{-1}$ and $\omega^k\sigma$ are diagonalizable with the same simple spectrum, the d-th roots of unity, so $h_k = d$. □

One frequency is the pairing rate, and all d frequencies span the full matrix algebra. The mask of an interval contains many frequencies (all of them when its length is prime to d), which is why interval masks needed Theorem 6.4.

Corollary 6.13 (Every address digit must be broken)**.** Lean (partial):
`two_mul_finrank_ker_sub_le_of_commute` [`Pigeonhole.lean`]
Not formalized: everything specific to the statement: that K commutes with T, the dimension count $\dim\ker[\tau, T] = d^2/q$, and the sharpness. Lean has only the abstract lemma.

Write the index set as $\mathbb{Z}/m \times \mathbb{Z}/q$, $d = mq$, so that the second coordinate is one digit of the address. Suppose every B_a has the form $B'_a \otimes C_a$ with all C_a in $F[G]$ for one non-derogatory $q \times q$ matrix G, for example with all C_a circulant. Then

$$\dim\ker(\tau - K) \;\le\; \tfrac{1}{2}\Big(d^2 + \frac{d^2}{q}\Big),$$

with equality for the partial transpose that exchanges the first digits of the two indices and leaves the second digits in place.

Proof (not formalized). K commutes with $T = 1 \otimes (1_m \otimes G)$, and as in Proposition 6.2 $\ker[\tau, T]$ is isomorphic to the centralizer of $1_m \otimes G$, which is $\mathrm{M}_m(F) \otimes F[G]$, of dimension $m^2 q = d^2/q$. Apply Lemma 6.1. The partial transpose is $\tau_m \otimes 1$ on $(F^m \otimes F^m) \otimes (F^q \otimes F^q)$; its second factors are $E_{u'u} \otimes 1_q$, and its swap code is $\mathrm{M}_m(F) \otimes \{\text{symmetric } q \times q \text{ matrices}\}$, of dimension $m^2 q(q+1)/2$. □

The transposition is the product of the commuting exchanges of the digits of the two addresses (Section 4.3). Corollary 6.13 says that a data path which treats any one digit of one address uniformly is at rate $\frac{1}{2} + \frac{1}{2q}$, however it treats the others; by Lemma 6.1, rate $1-\varepsilon$ needs $\operatorname{rank}[K, T] \ge (1 - 2\varepsilon - \frac{1}{q})d^2$ for the rotation T of every digit of either index. This is a requirement at every scale.

Remark 6.14 (Composition, and why this gives no logarithm)**.** Theorem 6.4 is a statement about one hop: the class-uniform rules with M classes are not closed under composition, and two of them compose to a rule with up to d classes. Lemma 6.1 does compose. Since $[K_2 K_1, T] = K_2[K_1, T] + [K_2, T]K_1$, $\operatorname{rank}[K_2 K_1, T] \le \operatorname{rank}[K_1, T] + \operatorname{rank}[K_2, T]$ (`finrank_range_commutator_comp_le`), and by induction

$$\operatorname{rank}[K_t \cdots K_1, T] \;\le\; \sum_i \operatorname{rank}[K_i, T].$$

For $T = 1 \otimes \sigma$ and a hop whose second factors lie in the span of the $P_J \sigma^e$ with M interval classes, $[K_i, T]X$ is supported on the M columns at which the class changes, so $\operatorname{rank}[K_i, T] \le Md$ and t such hops give $\dim\ker(\tau - K_t \cdots K_1) \le (d^2 + d + tMd)/2$.

The same subadditivity shows why Corollary 6.13 does not produce a factor $\log d$. An exact transposer needs $\sum_i \operatorname{rank}[K_i, T] \ge d^2 - d^2/q$ for the rotation T of every digit. But let U be the perfect unshuffle in base q on one index, which moves every digit to the next position and is a linear-time operation on tapes, and let G_j be the rotation of digit j of that index. Then $UG_j = G_{j+1}U$, so $[U, G_j] = (G_{j+1} - G_j)U$ has rank $d - d/q$, and $\operatorname{rank}[1 \otimes U, 1 \otimes G_j] = d^2 - d^2/q$ for every j at once: one operation supplies the whole requirement for all digits of that index. The requirement is real at every scale and can be met at all scales together.

6.5 What the theorem says

- *For translation-covariant rules, the half in the half bound is a pigeonhole.* A code larger than half the space meets its translate in at least $2D - d^2$ dimensions, and translation-covariant data has d dimensions per uniform piece. The argument does not depend on the characteristic, on the degree, on the window of the decoder, or on the other tensor side. It does depend on exact covariance under a unit shift inside each class, on the classes being the same for every output row, on linearity, and on there being one hop. The flag bounds of Section 5 are not shown to be instances of it.

- *No symmetry that the swap lacks.* By Lemma 6.1, a code that exceeds the pairing value by εd^2 needs $\operatorname{rank}[K, T] + \dim\ker[\tau, T] \geq d + 2\varepsilon d^2$ for every operator T. Together with Section 5: to beat the pairing rate, the decoder may have neither a hidden order nor a symmetry, on either index, that the swap does not share.

- *It uses the rule, not the relation.* The hypothesis of Theorem 6.4 is that translated receivers share a decoder. That is the feature of machines singled out in Section 4.6, and it is invisible in the cone relation.

- *The folklore step is optimal in dimension.* With M equal pieces, nothing beats the $M \times M$ coarse transposition by even one dimension, although many other codes equal it (Remark 6.7).

7 The construction side

This section says what the two theorems exclude, what they leave, and what would have to be added to make them statements about machines. Readings of the one-hop model as statements about tapes are labelled as readings.

7.1 Mechanisms that fail

Table 2 lists upper bounds on the code dimension for decoders with structure on one index, in the linear one-hop model, whatever is done on the other index. Each row is a theorem of Sections 5 and 6.

Structure on one index	Upper bound	By
At most two Kronecker terms	$d(d+1)/2$	Corollary 5.5
Commuting maps: any convolutions of any number of rows read in step	$d(d+1)/2$	Corollary 5.7
One p-group in characteristic p: one dyadic hierarchy over $\mathbb{F}_2$	$d(d+1)/2$	Corollary 5.7
All affine index maps, $d = p^m$, characteristic p	$d(d+1)/2$	Corollary 5.8
Expansion factor c along some flag	$(1 - \frac{1}{2c})d^2 + O(d)$	Corollary 5.9
Uniform decoders with M interval pieces, cycle or line	$(d^2 + Md)/2$	Theorems 6.4 and 6.9
Copy rules with M decoder classes of any shape	$(d^2 + Md)/2$	Proposition 6.8
s modulation frequencies	$\frac{s}{s+1}(d^2 + d)$	Corollary 6.12
One address digit of size q treated uniformly	$\frac{1}{2}(d^2 + d^2/q)$	Corollary 6.13

Table 2: Upper bounds on the code dimension in the linear one-hop model. The other index is arbitrary.

Two design rules follow, for the one-hop model.

- *Synchronous access is the pairing.* Reading any number of source rows in step with the output, at any offsets and through any convolution, gives the symmetric matrices and no more.
- *Uniform passes are the pairing.* A stretch on which translated outputs share their decoder gains nothing, whatever its window. The bound rises only with changes of regime, by $d/2$ symbols for each.

7.2 A reading for tapes

The following is a reading, not a theorem about machines. A pass in which the position read and the position written advance at the same speed, by a linear finite-state rule, computes a convolution with an unbounded window. On a line this is a two-sided rule in the sense of Theorem 6.9, with one class per stretch of unchanged regime, under two provisos: the regime changes at the same positions for every output row (change points that depend on the row need their common refinement), and the rule of each regime is applied to the whole stored word, with no state carried across a change. A state of q symbols carried across a change alters K by a rank of at most q per output row, so by Lemma 6.1 it costs a further term of order qd per change. Under the provisos, Theorem 6.9 bounds the gain over the pairing value by $d/2$ symbols per regime change, so rate $\frac{1}{2} + \delta$ needs at least $2\delta d$ regime changes, and for equal pieces the bound is attained by the block-symmetric matrices, that is, by the coarse step of the folklore recursion. Other speed ratios are decimations. On a cycle a single ratio is one twist class of Theorem 6.11; mixtures of ratios fall under Corollary 5.9 or Corollary 5.8.

This reading matches what the series proved about motion (Section 3.2): locally affine motion is excluded on the geometric side by the second and fourth papers, and here uniform decoding is excluded on the coding side, in a model that has no geometry at all. The seventh paper has a statement of the same kind for storage codes on relations: Cayley relations $\mathrm{Cay}(G, \{s, s^{-1}, t\})$ without transitive triangles give no gain over the pairing rate, for every finite group [9, Corollary 5.12]. Theorem 6.4 allows any degree and any window, but only one hop and only linear decoders.

7.3 What a linear-time machine would need

Combining Section 4.5 with Table 2: in addition to positive characteristic, curved motion, expanding cone relations and growing paired distance, a linear-time transposer, read in the one-hop model, needs on *both* indices a family of maps that

- has no invariant flag and no pair of flags with bounded expansion (Theorem 5.1);
- commutes with no non-derogatory translation up to $o(d)$ boundary columns, and intertwines none with boundedly many pairwise commuting others (Theorems 6.4 and 6.11);
- treats no address digit uniformly (Corollary 6.13).

The digit masks at every scale, which the folklore recursion uses, have these properties: their number grows with d, and no shift respects them. A bounded number of classes is not enough for rate near one, whatever their shape, because M classes leave a flag with $g(i) \le Mi$ and hence rate at most $1 - 1/(2M) + O(1/d)$. What the theorems leave open is how large a code a given number of classes can carry when the classes have no almost-period. It can exceed the value $(d^2 + Md)/2$ of every copy rule and of every interval rule, at $d = 6$ though not at $d = 5$ (Proposition 8.2 and Remark 8.1); whether a gain survives in rate is Problem 9.1.

7.4 From the one-hop model to machines

Three steps separate Theorem 6.4 from a statement about oblivious machines.

1. *A hypothesis on cone systems.* The coding bounds of the series use properties of the cone relation, and Section 3.5 shows that the relation-level properties proved so far cannot carry the conclusion. "Translated receivers share their decoder on every stretch of periodic head motion" is a property of the rule. It has to be stated for the cone systems of real schedules, where the receivers are output bits and the decoders are compositions of the transition function.

2. *Depth reduction.* The genie of Section 3.3 reveals values at chosen steps. The revealed word is advice that depends on position, so it breaks uniformity wherever it is used. A version of Theorem 6.4 with advice would have to charge each revealed bit as a regime change; Theorem 6.4 suggests the exchange rate, half a row per change, but proves nothing about it.

3. *Many hops.* Remark 6.14 bounds t uniform hops by a loss that grows linearly in t. A machine that runs for cL steps has unboundedly many hops, and Remark 6.14 shows that one cheap operation can break every digit symmetry at once. So an argument for machines has to count something that an unshuffle does not supply for free.

We do not know how to take any of these steps.

8 Small sizes

This section is numerical, except for Proposition 8.2, which is a formal statement about one explicit code. It reports exact computations over $\mathbb{F}_2$ for class-uniform rules (Definition 6.3) on a cycle. No proof in this paper uses them.

8.1 Method

For a partition of $\mathbb{Z}/d$ and a target dimension D, the existence of a class-uniform rule with a code of dimension at least D is a rank condition: the $d^2 \times d^2$ matrix of $\tau - K$, whose row (c, r) is determined by the functional of the class of c for the coordinate r, must have rank at most $d^2 - D$. We decide it in two ways.

- *Exhaustively*, for $d \le 4$, by enumerating subspaces and testing the criterion that a subspace $\mathcal{X}$ is a code if and only if, for every class J, $\sum_{c \in J} \sigma^c X_c = 0$ with $X_c \in \mathcal{X}$ implies $\sum_{c \in J}(\text{row } c \text{ of } X_c) = 0$ (`tools/exhaustive_pigeonhole_checks.py`).

- *By a SAT solver*, for $d = 5, 6$: the rank condition is written as a factorization of the matrix through $\mathbb{F}_2^{d^2-D}$, with the second factor in reduced echelon form, and given to CryptoMiniSat (`tools/class_uniform_sat.py`). A satisfying assignment is a rule together with its code; the script rebuilds the matrix from the rule, recomputes its kernel, and `tools/verify_class_uniform_code.py` checks the saved code against the definition, sharing no code with the search. An answer "unsatisfiable" rests on the solver, and we say so wherever we report one; as controls, the solver confirms the bound of Theorem 6.4 for $d = 4$ and for $\{0, 1\}, \{2, 3\}, \{4\}$ at $d = 5$. A time-out says nothing: the solver also times out on instances that are known to be satisfiable. Some searches were restricted to codes that contain the space $\mathcal{H}^*$ of matrices with $x_{c'} = \sigma^{c'-c} x_c$ for all c, c' in one class, which is itself a code of dimension Md and, for interval classes, is the space $\mathcal{H}$ of Section 6. This makes the search faster and can only lower the result. The code of Proposition 8.2 was found without the restriction and contains $\mathcal{H}^*$.

8.2 Results

Table 3 lists the largest dimension of a code for partitions of $\mathbb{Z}/d$, up to the affine maps $c \mapsto uc+t$ of the index. For $d \le 5$ every partition with $1 < M < d$ classes is listed; for $d = 6$, the partitions into two classes and the nine partitions into three classes for which the bound exceeds $(d^2 + Md)/2$. "Bound" is the best value of Theorem 6.4 over shift sets of at most four elements, rounded down. Every code counted in the last column for $d \ge 5$ is saved in `tools/data/`. The script `tools/small_sizes.py` prints the table from these files, with the bounds recomputed and every saved code checked again, and lists the partitions; `tools/README.md` records the runs of the solver, including those that timed out.

d	classes	pairing	$\lfloor (d^2 + Md)/2 \rfloor$	bound	largest code
3	$\{0\},\{1,2\}$	6	7	7	$= 7$
4	$\{0\},\{1,2,3\}$	10	12	12	$= 12$
	$\{0,1\},\{2,3\}$	10	12	12	$= 12$
	$\{0,2\},\{1,3\}$	10	12	12	$= 12$
	$\{0\},\{1\},\{2,3\}$	10	14	14	$= 14$
	$\{0\},\{2\},\{1,3\}$	10	14	14	$= 14$
5	$\{0\},\{1,2,3,4\}$	15	17	17	$= 17$
	$\{0,1\},\{2,3,4\}$	15	17	17	$= 17$
	$\{0\},\{1,2\},\{3,4\}$	15	20	20	$= 20$
	$\{0\},\{1\},\{2,3,4\}$	15	20	20	19^*
	$\{0\},\{1,4\},\{2,3\}$	15	20	21	20^*
	$\{0\},\{1\},\{2\},\{3,4\}$	15	22	22	$= 22$
6	$\{0\},\{1,\dots,5\}$	21	24	24	$= 24$
	$\{0,1\},\{2,\dots,5\}$	21	24	24	$= 24$
	$\{0,1,2\},\{3,4,5\}$	21	24	24	$= 24$
	$\{0,2,4\},\{1,3,5\}$	21	24	24	$= 24$
	$\{0,2\},\{1,3,4,5\}$	21	24	27	≥ 24
	$\{0,3\},\{1,2,4,5\}$	21	24	27	≥ 23
	$\{0,1,3\},\{2,4,5\}$	21	24	28	≥ 25
	$\{0,1\},\{2,4\},\{3,5\}$	21	27	30	≥ 28
	$\{0,1\},\{2,5\},\{3,4\}$	21	27	30	≥ 28
	$\{0,2\},\{1,4\},\{3,5\}$	21	27	30	≥ 28
	$\{0\},\{1,2,4\},\{3,5\}$	21	27	30	≥ 28
	$\{0\},\{1,5\},\{2,3,4\}$	21	27	30	≥ 27
	$\{0\},\{1,2,5\},\{3,4\}$	21	27	30	≥ 27
	$\{0\},\{1,3,4\},\{2,5\}$	21	27	30	≥ 27
	$\{0\},\{4\},\{1,2,3,5\}$	21	27	30	≥ 27
	$\{0\},\{3\},\{1,2,4,5\}$	21	27	30	≥ 26

Table 3: Largest codes of class-uniform rules over $\mathbb{F}_2$. "=" marks an exact value: a code of that dimension was found, and it equals the bound of Theorem 6.4. "$\ge$" marks the largest code found. A star marks a value that was found and whose optimality rests on the solver (Remark 8.1).

Remark 8.1 (Numerical: what the table shows)**.** (a) *Intervals.* For interval classes the bound $\lfloor (d^2 + Md)/2 \rfloor$ of Theorem 6.4 is attained in every case computed but one. For unequal pieces the codes that attain it are not block-symmetric, and at $d = 4$ with pieces $\{0\},\{1,2,3\}$ no copy rule attains it (Remark 6.7). The exception is $\{0\},\{1\},\{2,3,4\}$ at $d = 5$: a code of dimension 19 exists; by Lemma 6.5 a code of dimension 20 would contain $\mathcal{H}$; and the solver reports that no code of

dimension 20 contains $\mathcal{H}$. So the largest code has dimension 19, on the word of the solver, and the bound of Theorem 6.4 is not attained for every partition into intervals. (b) *No almost-period, $d = 5$.* For $\{0\}, \{1,4\}, \{2,3\}$ every single shift leaves four components, and the bound of Theorem 6.4 is 21, from a set of three shifts. A code of dimension $20 = (d^2 + Md)/2$ exists, and the solver reports that none of dimension 21 does. (c) *No almost-period, $d = 6$.* For $\{0,1,3\}, \{2,4,5\}$ every nonzero shift leaves four components, and the best bound of Theorem 6.4 is 28, from a set of three shifts. A code of dimension 25 exists, one more than $(d^2 + Md)/2 = 24$ (Proposition 8.2); searches for dimension 26 timed out. Four partitions into three classes have codes of dimension 28, one more than $(d^2 + Md)/2 = 27$. (d) The gain over $(d^2 + Md)/2$ in (c) is one dimension in each case. The data do not distinguish a gain of bounded size from one that grows.

Proposition 8.2 (Two classes with no almost-period beat every interval rule)**.** Lean: `exists_classUniformCode_gt`, `isClassUniformCode_aperiodicCode`, `finrank_aperiodicCode`, `aperiodicCode`, `aperiodicCls_eq_true_iff` [`AperiodicExample.lean`]

For $d = 6$ over $\mathbb{F}_2$ and the classes $\{0,1,3\}$ and $\{2,4,5\}$ of $\mathbb{Z}/6$ there is a code of a class-uniform rule of dimension 25. In particular the bound $(d^2 + Md)/2$ of Theorem 6.4 for M interval classes, and of Proposition 6.8 for copy rules with M classes of any shape, does not hold for coded rules on arbitrary classes.

Outline of the formal proof. The code is the span of 25 explicit matrices, and the two decoders are explicit (`AperiodicExampleData.lean`, generated from a certificate by `tools/gen_aperiodic_example.py`). That every generator satisfies the rule at every row, 900 equations over $\mathbb{F}_2$, is checked by the kernel (`aperiodicGen_row`); by linearity the span is a code (`isClassUniformCode_aperiodicCode`). Each generator has an entry equal to 1 where all the others vanish (`aperiodicGen_pivot`), so the generators are independent and the dimension is 25 (`finrank_aperiodicCode`). Then $2 \cdot 25 = 50 > 48 = 6^2 + 2 \cdot 6$ (`exists_classUniformCode_gt`). The certificate is used only as data. □

The code of Proposition 8.2 was found by the solver. It contains the 12-dimensional space $\mathcal{H}^*$, it meets its translate by one column in exactly $2 \cdot 25 - 36 = 14$ dimensions, the least that Lemma 6.1 allows, and it meets its translates by two and three columns in 18 and 17 dimensions (`tools/analyse_code.py`). We do not have a description of it that generalizes, and one example at $d = 6$ says nothing about rates: the term $Md/2$ is a quarter of $(d^2 + Md)/2$ at this size. What the example does show is that the union bound in Theorem 6.4 is not an artefact, and that masks without an almost-period carry a coding gain that no copy rule and no interval rule has. Problem 9.1 asks how large it can be.

9 Open problems

The question (T) is open. For oblivious machines it is the question about the function $r(\varepsilon)$ of Section 4.1. The problems below are the precise points at which this paper stops; the first three are finite or algebraic and can be attacked directly.

Problem 9.1 (Classes with no almost-period)**.** *For a partition $\mathcal{P}$ of $\mathbb{Z}/d$ into M classes let $D(\mathcal{P})$ be the largest dimension of a code of a class-uniform rule (Definition 6.3), over a given field. Theorem 6.4 gives $D(\mathcal{P}) \le (d^2 + Md)/2$ when the classes are intervals, and Proposition 8.2 shows that this fails for general classes.*

(a) For two classes, is $D(\mathcal{P}) \le (\frac{1}{2} + o(1))d^2$? More generally, is this so for $o(d)$ classes? The known upper bounds for two classes of any shape are $\frac{3}{4}d^2 + O(d)$, from Theorem 5.1, and what Theorem 6.4 gives for the best shift set.

(b) *Determine D for the classes $\{0,1,3\},\{2,4,5\}$ of $\mathbb{Z}/6$ over $\mathbb{F}_2$; it is at least 25 and at most 28.*

(c) *Find a family of partitions and codes, with a description that can be analysed, in which $D(\mathcal{P})-(d^2+Md)/2$ grows.*

A positive answer to (a) would be the half bound for all piecewise-uniform decoders in the one-hop model; it needs a count sharper than the union bound of Theorem 6.4, which loses a factor $s/(s+1)$ for $s+1$ translates. A negative answer would be a coding gain of positive rate native to masks and rotations, the first of its kind, and its cost on a tape would become the question. Section 8 records what exact computation says for small d.

Problem 9.2 (Unequal pieces)**.** *For which partitions of $\mathbb{Z}/d$ into M intervals, and over which fields, is the bound $\lfloor (d^2+Md)/2 \rfloor$ of Theorem 6.4 attained? It is attained for equal pieces (Proposition 6.6). Over $\mathbb{F}_2$ it is attained in every case computed in Section 8 except one, the pieces $\{0\},\{1\},\{2,3,4\}$ at $d=5$, where the largest code has dimension $19<20$ on the word of a solver. Some of the codes that attain it are not codes of copy rules.*

Problem 9.3 (Both sides, all scales)**.** *Let both the A_a and the B_a lie in the span of masks of dyadic intervals composed with rotations, with at most M_j intervals of length $d/2^j$ in use at scale j. Bound $\dim\ker(\tau-K)$ in terms of the profile $(M_j)_j$. The folklore recursion shows that the full profile suffices for dimension d^2; Theorem 6.4 and Corollary 6.13 treat one side and one scale at a time.*

Problem 9.4 (Shared decoders on cone systems)**.** *State and prove a version of Theorem 6.4 for the cone systems of oblivious linear-time schedules: on a stretch of periodic head motion the outputs share their decoder, and a genie that reveals εL bits changes the regime at most εL times. Does a bound of the form $r(\varepsilon)\le\frac{1}{2}+C(\varepsilon+\eta)$ hold for schedules whose motion is periodic outside a set of steps of density η? (Section 7.4.)*

Problem 9.5 (What an unshuffle does not supply)**.** *Corollary 6.13 requires a transposer to break the symmetry of every address digit, and Remark 6.14 shows that commutator rank cannot turn this into a factor $\log d$, because one unshuffle breaks all digits at once. Find a quantity that is subadditive over the steps of a machine, is large for the swap at every scale separately, and is not raised at all scales by a single linear-time operation. This is the operator form of the multi-scale problem of the sixteenth paper [12, Problem 11.1].*

Problem 9.6 (Relative rigidity on ball patterns)**.** *The eighth paper asks whether there is $\rho>0$ such that, for every linear-time schedule, every fixed D and all large d, the transposition matrix is not the sum of a matrix of rank below ρL and a matrix supported on the depth-D ball pattern of the schedule [2, Problem 10.6]; a positive answer alone would exclude oblivious linear-time transposers with linear data paths [3, Corollary 9.3]. The classes of this paper are defined by structure on one tensor side. Which ball patterns of schedules are contained in such a class, after a change of bases on one side? For those, Theorems 5.1 and 6.4 give $\rho=\frac{1}{2}-o(1)$.*

A Index of Lean declarations

Every declaration of this paper's library cited in the text, grouped by the Lean module that contains it, relative to the namespace `SwapRigidity`. All of them depend only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix B).

`SwapRigidity.AperiodicExample` `aperiodicCls_eq_true_iff`, `aperiodicCode`, `aperiodicGen_pivot`, `aperiodicGen_row`, `exists_classUniformCode_gt`, `finrank_aperiodicCode`, `isClassUniformCode_aperiodicCode`.

SwapRigidity.BlockSymmetric blockSymmetric, blockSymmetric_attains_bound, isClassUniformCode_blockSymmetric, two_mul_finrank_blockSymmetric.

SwapRigidity.ClassUniform IsClassUniformCode, IsClassUniformCode.finrank_le, IsClassUniformCode.row_eq_of_rot_mem, IsClassUniformCode.two_mul_finrank_le, IsClassUniformCode.two_mul_finrank_le_of_const, Link, rot.

SwapRigidity.Flag card_staircase, eq_zero_of_mem_swapCode_of_staircase, exists_staircase_finrank_eq, finrank_swapCode_conj, finrank_swapCode_le_of_flag, finrank_swapCode_le_of_flag_left, finrank_swapCode_le_of_staircase, finrank_swapCode_staircase, finrank_swapCode_swap, mem_swapCode, mem_swapCode_staircase_iff, staircaseRestrict_injective, swapCode, swapCodeEquiv, two_mul_finrank_swapCode_le_of_triangular, two_mul_finrank_swapCode_le_of_triangular_left, two_mul_sum_fin_succ.

SwapRigidity.Line IsLineUniformCode, IsLineUniformCode.row_succ_eq, IsLineUniformCode.two_mul_finrank_le, isLineUniformCode_of_coeff, isLineUniformCode_symmetric, lineView, truncShift.

SwapRigidity.Pigeonhole card_mul_finrank_add_le_iInf_comap, finrank_ker_commutator_sub_le, finrank_range_commutator_comp_le, finrank_range_commutator_le, mul_finrank_le_add_finrank_iInf_comap, two_mul_finrank_ker_sub_le, two_mul_finrank_ker_sub_le_ker, two_mul_finrank_ker_sub_le_of_commute.

SwapRigidity.Swap card_le_finrank_ker_swap_commutator, two_mul_finrank_ker_swap_sub_one, two_mul_finrank_range_swap_sub_one.

SwapRigidity.Twist finrank_le_sum_finrank_twistSpace, succ_mul_finrank_swapCode_le, succ_mul_finrank_swapCode_le_of_twist_classes, twistSpace, two_mul_finrank_swapCode_le_of_twist.

B Building and verifying the formalization

The Lean project is the directory `lean/` next to this paper. It uses Lean and Mathlib, both at version v4.34.1 (`lean-toolchain`, `lakefile.toml`, `lake-manifest.json`), and requires the shared `lean-machine/` library of the repository through a relative path, which supplies Mathlib at the pinned revision. It uses no library of an earlier paper. Keep the repository's directory layout so that `lean-machine/` stays in place.

Build. With `elan` installed, run in `lean/`:

```
lake exe cache get && lake build
```

The first command downloads the compiled Mathlib cache; the second compiles `SwapRigidity`.

Axioms and index. The file `lean/PaperAxioms.lean`, generated from the LaTeX sources of this paper by `tools/lean_index.py`, runs `#print axioms` on every declaration cited in this paper (Appendix A):

```
lake env lean PaperAxioms.lean
```

Every declaration depends at most on `propext`, `Classical.choice` and `Quot.sound`. No Lean file of this library uses `sorry`, `admit` or `native_decide`, declares an `axiom`, bypasses the kernel via `unsafe`, `implemented_by` or `extern`, or raises `maxHeartbeats`. Running `python3 tools/lean_index.py` from the paper's root directory regenerates both `paper/sections/leanindex.tex` and `lean/PaperAxioms.lean` (`-index-only` writes only the index; `-check` verifies that every cited name resolves and that both generated files are current).

Where things are. The library `lean/SwapRigidity/` has the following modules, all imported by the root module `lean/SwapRigidity.lean`.

- `Flag.lean` (Section 5): the swap code, the elimination on staircase patterns, invariance under changes of bases, the coordinate form of Theorem FL, the corollary for a pair of flags, the exchange of sides, and attainment.
- `Pigeonhole.lean` (Section 6): the pigeonhole lemma in its three forms, the rank bound for commutators and its subadditivity under composition, and the dimension count for an intersection of preimages.
- `Swap.lean`: the dimensions of the kernel and image of $\tau - 1$ and the bound $\dim \ker[\tau, T] \geq d$.
- `ClassUniform.lean`: class-uniform codes and Theorem TP on a cycle.
- `BlockSymmetric.lean`: the block-symmetric code, its dimension, and attainment of the bound.
- `Line.lean`: two-sided rules on a line and the line form of Theorem TP.
- `Twist.lean`: the twist-class theorem.
- `AperiodicExample.lean` and the generated `AperiodicExampleData.lean` (Proposition 8.2): an explicit class-uniform code of dimension 25 for $d = 6$ over $\mathbb{F}_2$. The data file contains definitions only. The finite facts about the data are proved by `decide`, which the kernel checks; nothing is taken on trust from the certificate `tools/data/aperiodic_d6_dim25.json` or from the generator `tools/gen_aperiodic_example.py` (run it with `-check` to confirm that the data file is current).

Computations. Every script that was used for this paper is in `tools/`, and the saved codes are in `tools/data/`. The file `tools/README.md` lists the scripts with their commands and the versions used, and records the solver runs behind Table 3, including those that timed out. Each script documents its usage at its top.

- `tools/exhaustive_flag_checks.py` and `tools/exhaustive_pigeonhole_checks.py` (Python with NumPy): the exhaustive and randomized checks behind the numerical remarks of Sections 5 and 6 and the entries for $d \leq 4$ of Table 3.
- `tools/three_term_example.py`: the examples of Remark 5.6.

- `tools/digit_and_composition_checks.py`: small instances of Corollary 6.13 and of Remark 6.14, both of which are proved in the text.
- `tools/class_uniform_sat.py` (needs the package `pycryptosat`): one run of the solver, for a partition and a target dimension.
- `tools/small_sizes.py`: Table 3 with its bounds, recomputed, and with every saved code checked again; the list of partitions up to affine maps; the search that raises the target dimension, which produced the saved codes; and controls that repeat the answers of the solver quoted in Remark 8.1.
- `tools/verify_class_uniform_code.py`: checks every saved code in `tools/data/` against the definition.
- `tools/analyse_code.py`: the intersections of the code of Proposition 8.2 with its translates, quoted after that proposition.

Apart from the certificate of Proposition 8.2, which Lean checks, these computations are not part of the formal development, and no proof depends on them.

References

[1] Micah Adler, Nicholas J. A. Harvey, Kamal Jain, Robert D. Kleinberg, and April Rasala Lehman. On the capacity of information networks. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2006)*, pages 241–250, 2006.

[2] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded nesting and depth reduction for matrix transposition on tapes. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Bounded-Nesting-and-Depth-Reduction-for-Matrix-Transposition-on-Tapes.

[3] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded-stack DAGs reduce to constant depth. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Bounded-Stack-DAGs-Reduce-to-Constant-Depth.

[4] Hans Brende and Claude Opus 5.5 (Anthropic). Curved motion, depth reduction, and storage codes for matrix transposition on tapes. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes.

[5] Hans Brende and Claude Opus 5.5 (Anthropic). Frozen gap areas and the sharpness of source-cover certificates. Memo to the ninth paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Source-Covers-and-Barriers-to-Tape-Transposition/memo-gap-area.

[6] Hans Brende and Claude Opus 5.5 (Anthropic). Lück approximation fails in positive characteristic. Manuscript and partial Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Luck-Approximation-Fails-in-Positive-Characteristic.

[7] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[8] Hans Brende and Claude Opus 5.5 (Anthropic). Merge depth, input steering, and the coding core of matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Merge-Depth-Input-Steering-and-the-Coding-Core-of-Matrix-Transposition-on-Tapes.

[9] Hans Brende and Claude Opus 5.5 (Anthropic). Nested elimination and half-rate bounds for storage codes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Nested-Elimination-and-Half-Rate-Bounds-for-Storage-Codes.

[10] Hans Brende and Claude Opus 5.5 (Anthropic). The padding theorem needs no idle tapes. Memo to the second paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Local-Coding-Barriers-and-the-Limits-of-Tape-Geometry-for-Matrix-Transposition/memo.

[11] Hans Brende and Claude Opus 5.5 (Anthropic). Storage codes above half rate under paired-distance separation. Manuscript and partial Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Storage-Codes-above-Half-Rate-under-Paired-Distance-Separation.

[12] Hans Brende, Claude Opus 5.5 (Anthropic), and Gemini 4 Argon (Google DeepMind). Oblivious linear-time tape transposers need macroscopic crossings. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Oblivious-Linear-Time-Tape-Transposers-Need-Macroscopic-Crossings.

[13] Hans Brende and Gemini 4 Argon (Google DeepMind). Directed reciprocity and deep-nesting bounds for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Directed-Reciprocity-and-Deep-Nesting-Bounds-for-Matrix-Transposition-on-Tapes.

[14] Hans Brende and GPT-6 Astra (OpenAI). Adaptive pass bounds without a runtime assumption. Memo to the first paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Matrix-Transposition-and-the-Tape-Coding-Conjecture/memo-passes.

[15] Hans Brende and GPT-6 Astra (OpenAI). Bit and word transposition on tapes. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Bit-and-Word-Transposition-on-Tapes.

[16] Hans Brende and GPT-6 Astra (OpenAI). Conditional elimination and circular storage codes. Memo to the fourth paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes/memo.

[17] Hans Brende and GPT-6 Astra (OpenAI). Covers and local generation in linear storage codes. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Covers-and-Local-Generation-in-Linear-Storage-Codes.

[18] Hans Brende and GPT-6 Astra (OpenAI). Directed recovery structures in tape computations. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Directed-Recovery-Structures-in-Tape-Computations.

[19] Hans Brende and GPT-6 Astra (OpenAI). High-rate separated storage codes and tape transposition. Manuscript and partial Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/High-Rate-Separated-Storage-Codes-and-Tape-Transposition.

[20] Hans Brende and GPT-6 Astra (OpenAI). Local coding barriers and the limits of tape geometry for matrix transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Local-Coding-Barriers-and-the-Limits-of-Tape-Geometry-for-Matrix-Transposition.

[21] Hans Brende and GPT-6 Astra (OpenAI). Regular-recursion obstructions for the five cascade schedules. Memo to the sixteenth paper of the series, with partial Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Oblivious-Linear-Time-Tape-Transposers-Need-Macroscopic-Crossings/memo-cascade-obstruction.

[22] Hans Brende and GPT-6 Astra (OpenAI). Soft recovery and record capacities in nested elimination. Memo to the seventh paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Nested-Elimination-and-Half-Rate-Bounds-for-Storage-Codes/memo-soft-recovery.

[23] Hans Brende and GPT-6 Astra (OpenAI). Source covers and barriers to tape transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Source-Covers-and-Barriers-to-Tape-Transposition.

[24] Hans Brende and GPT-6 Astra (OpenAI). Subcubic storage bounds beyond scalar recovery. Memo to the fourth paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes/memo-subcubic.

[25] Hans Brende and GPT-6 Astra (OpenAI). TCP and information-cheap depth reduction. Memo to the fourth paper of the series, written proof, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes/memo-information-depth.

[26] Chun-Yueh Chiang. A note on the $\top$-Stein matrix equation. *Abstract and Applied Analysis*, 2013, 2013. Article ID 824641; arXiv:1305.4243. doi:10.1155/2013/824641.

[27] Chun-Yueh Chiang, Eric King-Wah Chu, and Wen-Wei Lin. On the $\star$-Sylvester equation $AX \pm X^\star B^\star = C$. *Applied Mathematics and Computation*, 218(17):8393–8407, 2012. doi:10.1016/j.amc.2012.01.065.

[28] Fernando De Terán and Froilán M. Dopico. The solution of the equation $XA + AX^T = 0$ and its application to the theory of orbits. *Linear Algebra and its Applications*, 434(1):44–67, 2011. doi:10.1016/j.laa.2010.08.005.

[29] Fernando De Terán, Froilán M. Dopico, Nathan Guillery, Daniel Montealegre, and Nicolás Reyes. The solution of the equation $AX + X^\star B = 0$. *Linear Algebra and its Applications*, 438(7):2817–2860, 2013. doi:10.1016/j.laa.2012.11.014.

[30] Fernando De Terán, Bruno Iannazzo, Federico Poloni, and Leonardo Robol. Solvability and uniqueness criteria for generalized Sylvester-type equations. *Linear Algebra and its Applications*, 542:501–521, 2018. arXiv:1608.01183. Corrigendum: ibid., pp. 522–526. doi:10.1016/j.laa.2017.07.010.

[31] Martin Dietzfelbinger and Wolfgang Maass. The complexity of matrix transposition on one-tape off-line Turing machines with output tape. *Theoretical Computer Science*, 108(2):271–290, 1993.

[32] Martin Dietzfelbinger, Wolfgang Maass, and Georg Schnitger. The complexity of matrix transposition on one-tape off-line Turing machines. *Theoretical Computer Science*, 82(1):113–129, 1991.

[33] Étienne Grandjean. A natural NP-complete problem with a nontrivial lower bound. *SIAM Journal on Computing*, 17(4):786–809, 1988. doi:10.1137/0217050.

[34] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. arXiv:2503.22848; also in Proc. FOCS 2025, pp. 1366–1381 (numbering as in arXiv), 2025. URL: https://arxiv.org/abs/2503.22848.

[35] Walter W. Kirchherr. Transposition of an $l \times l$ matrix requires $\Omega(\log l)$ reversals on conservative Turing machines. *Information Processing Letters*, 28(2):55–59, 1988. doi:10.1016/0020-0190(88)90163-9.

[36] Jan R. Magnus and Heinz Neudecker. The commutation matrix: Some properties and applications. *The Annals of Statistics*, 7(2):381–394, 1979. doi:10.1214/aos/1176344621.

[37] OpenAI. Integer multiplication below $n \log n$. GitHub release, repository `openai/math`, family 109, September 2026. Preprint dated 23 September 2026. URL: https://github.com/openai/math/blob/main/preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf.

[38] Wolfgang J. Paul, Nicholas Pippenger, Endre Szemerédi, and William T. Trotter. On determinism versus non-determinism and related problems. In *24th Annual Symposium on Foundations of Computer Science (FOCS 1983)*, pages 429–438, 1983.

[39] Søren Riis. Information flows, graphs and their guessing numbers. *Electronic Journal of Combinatorics*, 14(1):Research Paper R44, 2007. doi:10.37236/962.

[40] Hans-Jörg Stoß. Rangierkomplexität von Permutationen. *Acta Informatica*, 2:80–96, 1973.

[41] Hanqi Tang, Qifu Tyler Sun, Zongpeng Li, Xiaolong Yang, and Keping Long. Circular-shift linear network coding. *IEEE Transactions on Information Theory*, 65(1):65–80, 2019. doi:10.1109/TIT.2018.2832624.

[42] Leslie G. Valiant. Graph-theoretic arguments in low-level complexity. In *Mathematical Foundations of Computer Science (MFCS)*, volume 53 of *Lecture Notes in Computer Science*, pages 162–176. Springer, 1977.